

PANTHER: Robust Hybrid KEM Combiners via Structural Splicing

Basker Palaniswamy^{1*} , Paolo Palmieri¹  & Ashok Kumar Das^{2,3} 

¹Insight Research Ireland Centre for Data Analytics,

Department of Computer Science and Information Technology,

University College Cork (UCC), Cork City, Ireland, European Union.

²Center for Security, Theory and Algorithmic Research,

International Institute of Information Technology, Hyderabad, 500 032, India.

³Department of Computer Science and Engineering, College of Informatics,

Korea University, 145 Anam-ro Seongbuk-gu, Seoul, 02841, South Korea.

{1* - basker170889@gmail.com, 1 - paolo.palmieri@ucc.ie,}

{2,3 - iitkgp.akdas@gmail.com, ashok.das@iiit.ac.in}

*Corresponding Author

Abstract. We present *Panther*, a family of six robust hybrid key encapsulation mechanism (KEM) combiners that pair FrodoKEM (unstructured LWE) with ML-KEM (module-LWE, FIPS 203) so that IND-CCA2 security holds whenever either assumption is hard. The family includes five hardened variants of the textbook combiners—parallel HKDF, SHAKE256 split-key, sequential chaining, XOR, and nested—each made to satisfy a uniform robustness predicate (transcript binding, domain separation, implicit rejection, length normalisation, $\vee$ -security), together with a novel structural-splicing construction *Panther-SS* that interleaves the constituent ciphertexts and binds the cut-positions via a structural tag. Every combiner admits a systematic Market-Theoretic Security Framework proof in which each bidding round is documented by its purpose, the scheme component it replaces, and its complexity cost; the framework extends cleanly to correctness, unbounded session security, QROM security, and quantitative side-channel resistance.

We complement the theory with extensive benchmarks on liboqs-backed reference implementations, including a head-to-head comparison of *Panther* combiners against the key-encapsulation candidates that appeared in NIST PQC Rounds 1–4 (Kyber/ML-KEM, FrodoKEM, NTRU, SABER, NTRU Prime, Classic McEliece, BIKE, HQC). The experiments cover key-gen/encaps/decaps latency, throughput, memory footprint, ciphertext and key sizes, scaling with query count, CPU-cycle counts, security-vs-performance Pareto analysis, and an attack-vs-defence matrix against published side-channel attacks on both constituents. The results confirm that hybrid robustness is essentially free over the slower constituent, that *Panther-SS* uniquely achieves full robustness with combiner-only overhead below half a percent of total latency, and that the *Panther* family sits on the Pareto frontier of post-quantum KEM candidates. We believe TLS 1.4 could benefit from this work.

Keywords: Hybrid KEM · Post-quantum cryptography · ML-KEM · FrodoKEM · IND-CCA2 · MTSF · Bidding-round proofs · QROM · Structural splicing · Transcript binding · Implicit rejection · Side-channel resistance · NIST PQC benchmarking · TLS 1.4.

Table of Contents

PANTHER: Robust Hybrid KEM Combiners via Structural Splicing	1
<i>Basker Palaniswamy^{1*} , Paolo Palmieri¹  & Ashok Kumar Das^{2,3} </i> ¹ <i>Insight Research Ireland Centre for Data Analytics, Department of Computer Science and Information Technology, University College Cork (UCC), Cork City, Ireland, European Union. ²Center for Security, Theory and Algorithmic Research, International Institute of Information Technology, Hyderabad, 500 032, India. ³Department of Computer Science and Engineering, College of Informatics, Korea University, 145 Anam-ro Seongbuk-gu, Seoul, 02841, South Korea. {1* - basker170889@gmail.com, 1 - paolo.palmieri@ucc.ie,}</i> <i>{2,3 - iitkgp.akdas@gmail.com, ashok.das@iiit.ac.in} *Corresponding Author</i>	
1 Introduction	4
1.0.1 Why hybrid KEMs?	4
1.0.2 The combiner problem.	4
1.0.3 Our contributions.	4
1.0.4 Structure of the paper.	5
2 Related Work	5
2.0.1 Post-quantum KEMs.	5
2.0.2 Hybrid KEM combiners.	5
2.0.3 Key derivation and transcript binding.	5
2.0.4 Security frameworks.	5
2.0.5 Implementation-level attacks.	6
3 Preliminaries	6
3.1 Notation	6
3.2 Key Encapsulation Mechanisms	6
3.3 Lattice Assumptions	7
3.4 Fujisaki–Okamoto and the QROM	7
3.5 The Market-Theoretic Security Framework	7
3.6 Adversary Model and Attacker Taxonomy	8
3.7 Primitive Parameters	9
4 The Panther Family of Robust Hybrid KEM Combiners	10
4.1 Design Goals and Robustness Requirements	10
4.2 Common KeyGen	10
4.3 Panther-P: Robust Parallel Concatenation with KDF	11
4.4 Panther-S: Robust Keccak-Based Split-Key PRF	11
4.5 Panther-C: Robust Sequential Chaining	12
4.6 Panther-X: Robust XOR Combiner	12
4.7 Panther-N: Robust Nested KEM	13
4.8 Panther-SS: Structural Splicing (New Construction)	13
4.9 Summary of Combiners	15
5 Novelty, Positioning, and Comparison with Prior Work	16
5.1 Summary of Novel Contributions	16
5.2 Comparison with Prior Hybrid KEM Combiners	16
5.3 Why Additive Pricing Beats Minimum Pricing	16

5.4	Relationship to MTSF	17
6	Security Analysis	17
6.1	Correctness of Panther Combiners	18
6.2	Master Splitting Lemma for $\vee$ -Security	18
6.3	Security of Panther-P	19
6.4	Security of Panther-S	20
6.5	Security of Panther-C	21
6.6	Security of Panther-X	22
6.7	Security of Panther-N	23
6.8	Security of Panther-SS: Structural Splicing	23
6.9	Unbounded Session Security	26
6.10	Post-Quantum Security in the QROM	27
6.11	Summary of Security Bounds	28
7	Side-Channel Attack Resistance of Panther	29
7.1	Side-Channel Vulnerabilities of Stand-Alone ML-KEM and FrodoKEM	29
7.2	How Panther Design Choices Block Each Attack	30
7.3	Attack-vs-Defence Matrix	31
7.4	Quantitative Leakage Comparison	31
7.5	Summary and Deployment Implications	32
8	Experimental Study	33
8.1	Methodology	33
8.2	Keygen, Encaps, and Decaps Timings	34
8.3	Combiner-Only Overhead	34
8.4	Stand-alone versus Hybrid: Total Latency Comparison	35
8.5	Throughput	35
8.6	Ciphertext and Key Sizes	36
8.7	Memory Footprint	37
8.8	Scaling with Decapsulation Query Count	37
8.9	Latency Distribution	37
8.10	Security-vs-Performance Trade-Off	37
8.11	Rigorous Benchmarking Against NIST PQC Candidates	38
8.12	Discussion	42
9	Conclusion	42
	Acknowledgements	43
	References	43
	Index	51

1 Introduction

The standardisation of post-quantum key encapsulation by the US National Institute of Standards and Technology (NIST) culminated in 2024 with the publication of FIPS 203 [31], which specifies ML-KEM (derived from CRYSTALS-Kyber [11]) as a module-lattice-based mechanism offering compact keys and ciphertexts at strong post-quantum security levels. The NIST process also retained FrodoKEM, an unstructured-lattice construction based on plain Learning-With-Errors (LWE) [38,10,3], as a conservative alternative whose security does not rely on the algebraic structure of modules or ideals.

1.0.1 Why hybrid KEMs? Real-world deployments rarely trust a single hardness assumption. The ongoing migration of TLS, IPsec/IKEv2, SSH, and messaging protocols to post-quantum cryptography is therefore happening in *hybrid* mode: a classical primitive (e.g. ECDH) or a conservative post-quantum primitive (e.g. FrodoKEM) is combined with a structured-lattice KEM (e.g. ML-KEM), so that the session key remains secure whenever *at least one* of the two constituents is secure [8,7,41,43,13]. Hybridisation diversifies assumption risk and cushions against future cryptanalytic surprises.

1.0.2 The combiner problem. A *combiner* takes the ciphertexts and shared secrets of two KEMs KEM_1 and KEM_2 and produces a single hybrid ciphertext ct_h and shared secret ss_h . The design space is well trodden but uneven. Five textbook combiners appear repeatedly in standardisation drafts and academic prototypes:¹ (i) *parallel concatenation* with a KDF; (ii) *Keccak-based split-key derivation* using SHAKE256; (iii) *sequential chaining* with iterative PRF updates; (iv) *XOR combination* of the shared secrets; (v) *nested* encapsulation. Each has known weaknesses. Plain XOR lacks transcript binding and silently accepts malformed ciphertexts; sequential chaining adds latency and complicates reproducibility; naive concatenation ignores domain separation, opening cross-protocol and cross-construction confusion [18]; nested composition inherits the worst of both assumptions and destroys the IND-CCA2 diversification guarantee.

1.0.3 Our contributions. This paper makes four contributions.

1. *Robust Panther combiners.* We present a family $\{\text{Panther-P, Panther-S, Panther-C, Panther-X, Panther-N}\}$ of the five combiners, each systematically hardened with explicit transcript binding, domain separation, equal-length normalisation (to defeat XOR-malleability), implicit rejection (to match the ML-KEM IND-CCA2 profile), and constant-time decapsulation. Each construction is defined in Section 4 as an explicit algorithm with all intermediate values named.
2. *Structural splicing (Panther-SS).* We introduce a new combiner that interleaves the two constituent ciphertexts and key-material inside a spliced-transcript key derivation. Unlike XOR or nested encapsulation, Panther-SS preserves the $\vee$ -security property—IND-CCA2 holds whenever *either* MLWE or LWE is hard—while simultaneously binding the final shared secret to the full transcript and enforcing per-session novelty via a structural tag.
3. *Unified MTSF security analysis.* We apply the recently introduced Market-Theoretic Security Framework [33] to every Panther combiner. Each proof is a sequence of bidding rounds $\mathbf{B}_0 \rightarrow \mathbf{B}_q$ in which each hop is priced by an explicit failure event (implicit-rejection bypass, decapsulation correctness error, KDF distinguisher, XOR-malleability event, $\dots$). For each construction we obtain a concrete ask-price bound $\text{Ask}(\text{g}_{\text{IND-CCA2}}) \leq \sum_k \Delta\text{Price}_k + \text{negl}(\lambda)$. We also extend the analysis to

¹ We summarise the five combiners in Section 4. Our terminology follows [8,41,43].

unbounded encapsulation sessions using session pinging and CNF session verification [33], giving a linear accumulation bound $\text{Ask}_i \leq \epsilon + (i - 1)\delta_{\text{ping}}$ that remains negligible for any polynomial number of sessions.

4. *Experimental study.* We evaluate each combiner using liboqs-backed reference implementations of FrodoKEM-976 and ML-KEM-768 [32,42], measuring keygen, encaps, and decaps latencies; ciphertext, public-key, and shared-secret sizes; and combiner-specific overheads. The results show that the most robust combiner (Panther-SS) costs less than 0.5% over Panther-P and 1.3% over the minimal Panther-X.

1.0.4 Structure of the paper. Section 2 surveys related work on hybrid KEMs, combiners, and security frameworks. Section 3 fixes notation and recalls the MTSF bidding-round machinery. Section 4 presents the six Panther constructions. Section 6 gives the security analysis. Section 8 reports experimental results. Section 9 concludes.

2 Related Work

2.0.1 Post-quantum KEMs. Lattice-based public-key encryption was introduced by Ajtai and Dwork [1] and matured with Regev’s LWE problem [38], its ring variant [27], and the module-LWE problem [26]. ML-KEM (formerly CRYSTALS-Kyber; now FIPS 203 [31,11]) uses the Fujisaki–Okamoto transformation [17,20] on a module-LWE PKE, with IND-CCA2 security proven both in the ROM and the QROM [20,9,44,14]. FrodoKEM [10,3] uses plain LWE with no algebraic structure at the cost of multi-kilobyte ciphertexts, and is positioned as a conservative hedge. NIST’s status reports [2] document the reasoning behind retaining both structured and unstructured alternatives.

2.0.2 Hybrid KEM combiners. Giacomelli, Madec, and Smart [18] formalise combiners in the backdoored-random-oracle setting. Bindel et al. [8,7] introduce a family of N -of- N combiners and analyse hybrid authenticated key-exchange. IETF has standardised combiner use in IKEv2 (RFC 9370 [43]) and in the TLS 1.3 hybrid-design draft [41]. Crockett, Paquin, and Stebila [13,34] demonstrated prototype deployment in TLS and SSH. Schanck and Stebila [39] earlier considered NTRU-based hybrid constructions. Our work synthesises this thread: we adopt the FrodoKEM+ML-KEM pairing favoured by NIST [2] and the hardening techniques of [41,43], but unify their analyses under a single proof methodology.

2.0.3 Key derivation and transcript binding. Krawczyk’s HKDF [25,24] provides a principled extract-then-expand construction that is a natural vehicle for transcript binding. NIST SP 800-56C [30] and SP 800-185 [23] standardise Keccak-based two-step derivations. FIPS 202 [29] specifies SHAKE256, which is the core primitive of our Panther-S and Panther-SS combiners. Bellare, Canetti, and Krawczyk [4] pioneered keyed-hash MACs, whose construction influenced all subsequent PRF-based combiner designs.

2.0.4 Security frameworks. Bellare and Rogaway [5,6] established the random-oracle methodology and the code-based game-playing paradigm. Shoup [40] popularised game-hopping as the standard proof structure. Canetti [12] introduced universal composability. In the post-quantum setting, Boneh et al. [9] formalised the quantum random-oracle model (QROM); Unruh’s one-way-to-hiding lemma [44]

and the measure-and-reprogram technique of Don, Fehr, Majenz, and Schaffner [14] provide the QROM analogues of classical reprogramming arguments. Dowling, Fischlin, Günther, and Stebila [15] analyse TLS 1.3 under game-based methodology. The Market-Theoretic Security Framework (MTSF) [33] unifies these strands by framing each game hop as a *bidding round* $\mathbf{B}_k$ with an associated *price adjustment* ΔPrice_k ; the total adversarial advantage equals the ask-price of the security good and is bounded by $\sum_k \Delta\text{Price}_k$. MTSF additionally supplies a CNF-based session-verification procedure and a session-pinging argument that extends bounded-session security to unbounded sessions. We adopt MTSF as our uniform proof vehicle.

2.0.5 Implementation-level attacks. Side-channel attacks on lattice KEMs have become a central concern. Guo, Johansson, and Nilsson [19] describe key-recovery timing attacks on FO-transformed KEMs including FrodoKEM. Ravi et al. [37] give generic side-channel attacks on CCA-secure lattice KEMs. Any realistic combiner must therefore *preserve the implicit-rejection profile* of each constituent, which is one of the explicit hardening requirements we impose on every **Panther** construction in Section 4. The classical d -probing model [21] underlies the side-channel analysis we refer to in our security discussion.

3 Preliminaries

3.1 Notation

We denote the security parameter by λ . A function $f: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible*, written $f = \text{negl}(\lambda)$, if for every polynomial p and all sufficiently large λ we have $f(\lambda) \leq 1/p(\lambda)$. We write $x \xleftarrow{\$} S$ for sampling x uniformly from a finite set S and $y \leftarrow A(x)$ for the (deterministic or randomised) output of an algorithm A on input x . All adversaries are probabilistic polynomial-time (PPT) Turing machines unless stated otherwise.

Byte-string concatenation is written $x \parallel y$. For a bit-string $x \in \{0, 1\}^n$ we write $|x| = n$. Equality of bit-strings is denoted $=$; constant-time equality is denoted $\stackrel{?}{=}_{\text{ct}}$.

3.2 Key Encapsulation Mechanisms

Definition 1 (KEM). A key encapsulation mechanism is a triple of PPT algorithms $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$:

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$: generates a public/private key pair.
- $(\text{ct}, \text{ss}) \leftarrow \text{Encaps}(\text{pk})$: outputs a ciphertext ct and a shared secret $\text{ss} \in \{0, 1\}^\ell$.
- $\text{ss} \leftarrow \text{Decaps}(\text{sk}, \text{ct})$: recovers the shared secret, or returns $\perp$.

KEM is δ -correct if $\Pr[\text{Decaps}(\text{sk}, \text{ct}) \neq \text{ss} \mid (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}, (\text{ct}, \text{ss}) \leftarrow \text{Encaps}(\text{pk})] \leq \delta$.

Definition 2 (IND-CCA2 for KEMs). For a KEM KEM , the IND-CCA2 game proceeds as follows:

1. Challenger runs $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$, $(\text{ct}^*, \text{ss}_0) \leftarrow \text{Encaps}(\text{pk})$, samples $\text{ss}_1 \xleftarrow{\$} \{0, 1\}^\ell$ and $b \xleftarrow{\$} \{0, 1\}$.
2. $\mathcal{A}$ is given $(\text{pk}, \text{ct}^*, \text{ss}_b)$ and a decapsulation oracle $\mathcal{O}_D(\cdot)$ that returns $\text{Decaps}(\text{sk}, \cdot)$ on any query $\text{ct} \neq \text{ct}^*$.
3. $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ and wins iff $b' = b$.

KEM is IND-CCA2 secure if $\text{Adv}_{\text{KEM}, \mathcal{A}}^{\text{IND-CCA2}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|$ is negligible for every PPT $\mathcal{A}$.

3.3 Lattice Assumptions

Definition 3 (LWE [38]). Let $n, m, q \in \mathbb{N}$ and let χ be an error distribution on $\mathbb{Z}_q$. The Learning-With-Errors (LWE) problem $\text{LWE}_{n,m,q,\chi}$ asks to distinguish, given matrices $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ sampled uniformly, whether $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ for $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$ and $\mathbf{e} \xleftarrow{\$} \chi^m$, or $\mathbf{b} \xleftarrow{\$} \mathbb{Z}_q^m$ uniform. The advantage $\text{Adv}^{\text{LWE}}(\lambda)$ is the distinguishing probability.

Definition 4 (MLWE [26]). Let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. The Module Learning-With-Errors problem $\text{MLWE}_{k,\ell,q,\chi}$ asks to distinguish $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ from uniform, where $\mathbf{A} \in R_q^{k \times \ell}$, $\mathbf{s} \xleftarrow{\$} R_q^\ell$, $\mathbf{e} \xleftarrow{\$} \chi^k$. The advantage is denoted $\text{Adv}^{\text{MLWE}}(\lambda)$.

FrodoKEM reduces to (a decisional variant of) LWE and ML-KEM to MLWE. Worst-case-to-average-case reductions [38,26,35] underpin the hardness hypotheses.

3.4 Fujisaki–Okamoto and the QROM

Both ML-KEM and FrodoKEM apply a variant of the FO transform [17,20] to a δ -correct IND-CPA PKE $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ in order to obtain IND-CCA2 security. With implicit rejection, the transform $\text{FO}^\chi[\text{PKE}, H, G]$ derives the encapsulated key as $K \leftarrow G(m \parallel c)$ on a correct decapsulation and $K \leftarrow G(s \parallel c)$ using a secret rejection seed s otherwise, making decapsulation deterministic and constant-time. In the QROM, the combination of the one-way-to-hiding lemma of Unruh [44] and the measure-and-reprogram technique of Don, Fehr, Majenz, and Schaffner [14] yields tight bounds of the form

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}} \leq 2(q_H + 1) \sqrt{\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}} + q_D \cdot \delta + \text{negl}(\lambda).$$

3.5 The Market-Theoretic Security Framework

We adopt the proof methodology of MTSF [33], which we summarise for self-containment. A security property (e.g. IND-CCA2) is modelled as a *good* g for sale by the *seller* (challenger) to the *buyer* (adversary). The *ask-price* $\text{Ask}(g)$ is the maximum adversarial advantage against the property. A proof proceeds as a sequence of *bidding rounds* $\mathbf{B}_0, \dots, \mathbf{B}_q$, where $\mathbf{B}_0$ is the real game and $\mathbf{B}_q$ is an ideal game in which the buyer advantage is zero. Each transition $\mathbf{B}_k \rightarrow \mathbf{B}_{k+1}$ has an associated *price adjustment*

$$\Delta \text{Price}_k = |\Pr[\mathbf{B}_k(\mathcal{A}) = 1] - \Pr[\mathbf{B}_{k+1}(\mathcal{A}) = 1]|.$$

Lemma 1 (Classical Difference Lemma [40,6]). If $\mathbf{B}_k$ and $\mathbf{B}_{k+1}$ are identical unless a single failure event F occurs, then $\Delta \text{Price}_k \leq \Pr[F \text{ in } \mathbf{B}_k]$.

Lemma 2 (Extended Difference Lemma [33]). If $\mathbf{B}_k$ and $\mathbf{B}_{k+1}$ are identical unless at least one of the events $F_1, \dots, F_m$ occurs, then

$$\Delta \text{Price}_k \leq \Pr[\bigcup_{i=1}^m F_i] \leq \sum_{i=1}^m \Pr[F_i] \quad (\text{union bound}), \quad (1)$$

with a tighter inclusion-exclusion bound available when correlations are known.

The total ask-price is the telescoping sum

$$\text{Ask}(g) \leq \sum_{k=0}^{q-1} \Delta \text{Price}_k. \quad (2)$$

Session correctness and unbounded security. MTSF equips every session of a protocol with a conjunctive-normal-form *session-CNF* $\varphi_i = \bigwedge_j \varphi_i^{(j)}$ that encodes correctness predicates (SID binding, nonce freshness, signature/MAC verification, ciphertext freshness). A *ping* procedure $\text{Ping}(\mathcal{S}_i, \mathcal{S}_{i+1})$ certifies inter-session structural distinctness. When the per-session advantage is bounded by ϵ and the ping failure probability is δ_{ping} , the unbounded-security theorem [33] yields $\text{Ask}_i \leq \epsilon + (i - 1)\delta_{\text{ping}}$ for all $i \geq 1$, which is negligible for any polynomial number of sessions when both ϵ and δ_{ping} are negligible. We invoke this theorem in Section 6.9 to extend the bounded IND-CCA2 security of each Panther combiner to unbounded encapsulation sessions.

Market opens: real game

Market closes: $\mathcal{A}$'s advantage = 0

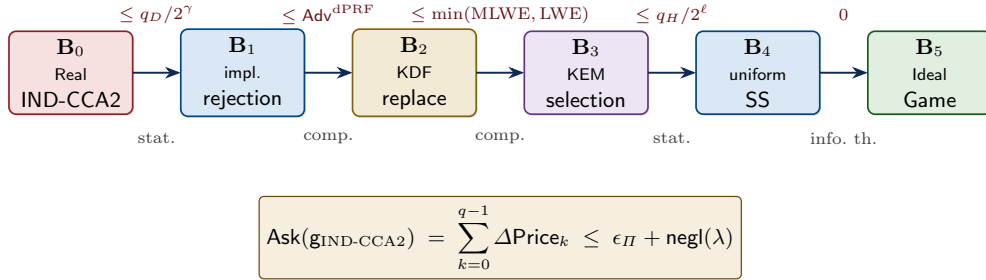


Fig. 1: **MTSF bidding-round architecture applied to Panther.** The buyer (adversary $\mathcal{A}$) plays a sequence of games $\mathbf{B}_0, \mathbf{B}_1, \dots, \mathbf{B}_q$; each hop *extracts a specific adversarial capability* and *prices* it either statistically, computationally (by reduction to MLWE/LWE), or information-theoretically. The telescoping sum of per-hop price adjustments is the ask-price of the IND-CCA2 security good—the adversary’s total winning advantage. Different Panther combiners instantiate this chain with different intermediate games; the common reduction template is the *Master Splitting Lemma* (Lemma 3).

3.6 Adversary Model and Attacker Taxonomy

We make the threat model explicit because one of our central claims is *robustness*: Panther combiners must resist not only the textbook IND-CCA2 adversary but also a spectrum of concrete attack strategies that arise whenever two KEMs are combined. We distinguish five adversary classes; every Panther construction in Section 4 is designed to defend against all of them, and every proof in Section 6 accounts for each as an explicit bidding round.

★ Intuition: Attack surface of a hybrid KEM

A hybrid KEM exposes *more* than the union of the two underlying attack surfaces. Attackers can target the constituent KEMs *individually*, the combiner *itself*, the *interface* between them (e.g. what is fed into the KDF), and the *protocol deployment* (e.g. which combiner is negotiated). A single design choice can close dozens of attack vectors at once; a subtle mistake can open new ones.

Definition 5 (Adversary Classes). *A PPT adversary $\mathcal{A}$ against a hybrid KEM KEM_h belongs to one or more of the following five classes. The taxonomy is additive: a real adversary may combine capabilities from several classes.*

Class A — Single-primitive adversary. *$\mathcal{A}$ targets exactly one of the two constituent KEMs (ML-KEM or FrodoKEM), treating the other as a black box. Captured by the standard IND-CCA2 games for each constituent.*

Class B — Combiner-level adversary. *$\mathcal{A}$ exploits weaknesses in the combiner’s KDF, transcript binding, or domain-separation machinery, without breaking either constituent. Captured by the KDF-replacement bids in Theorem 2–Theorem 7.*

Class C — Interface adversary. *$\mathcal{A}$ induces a mismatch between what Encaps feeds into the KDF and what Decaps can reconstruct from ciphertexts (e.g. length extension, splice-factor confusion, malformed chained seed). Captured by the transcript-binding and structural-tag bids ($B1 \rightarrow B2$ of Panther-SS).*

Class D — Cross-combiner adversary. *$\mathcal{A}$ exploits deployments that negotiate multiple combiners concurrently, trying to cross-feed ciphertexts from one combiner’s encapsulator into another’s decapsulator to produce a shared-secret collision. Defended against only by Panther-SS’s structural tag (Section 4.8).*

Class E — Side-channel adversary. *$\mathcal{A}$ observes timing, power, EM, cache-pattern, or induced faults on a specific implementation of either constituent. Covered in Section 7; Panther combiners provide defence-in-depth amplification even when each constituent is only partially hardened.*

✿ Novelty: Adversary classes vs combiner defences

No prior hybrid KEM analysis explicitly enumerates Classes C and D. Our bidding-round treatment assigns a concrete pricing hop to each class, making the coverage *auditable*: every combiner has a verifiable checklist of which classes it defends against (Table 2), and every proof has a named bidding round for each class. This traceability is what the Market-Theoretic Security Framework buys over ad-hoc hybrid analyses.

3.7 Primitive Parameters

Throughout the paper we instantiate FrodoKEM at NIST Level 3 (Frodo-976) and ML-KEM at the matching Level 3 (ML-KEM-768). The resulting parameter sizes are summarised in Table 1.

Table 1: Approximate sizes (bytes) of the two constituent KEMs at NIST Level 3. Values are drawn from the FIPS 203 standard [31] and the FrodoKEM round-3 specification [3].

KEM	Public key	Private key	Ciphertext	Shared secret
ML-KEM-768	1184	2400	1088	32
FrodoKEM-976	15 632	31 296	15 792	24

4 The Panther Family of Robust Hybrid KEM Combiners

Let $\text{KEM}_1 = \text{FrodoKEM}$ and $\text{KEM}_2 = \text{ML-KEM}$ be instantiated at compatible security levels. The shared-secret lengths of the two primitives are denoted ℓ_1 and ℓ_2 respectively (24 and 32 bytes at NIST Level 3). The target hybrid shared-secret length is $\ell_h = 32$ bytes unless otherwise stated.

4.1 Design Goals and Robustness Requirements

Every Panther combiner satisfies the following uniform design requirements, which we abbreviate as the **Panther robustness predicate P**:

(P1) Transcript binding.

The derivation of ss_h is a deterministic function of the entire hybrid ciphertext ct_h , both public keys $(\text{pk}_1, \text{pk}_2)$, and both constituent shared secrets; no input is dropped by the KDF.

(P2) Domain separation.

The derivation includes a scheme-specific domain-separation label and a construction tag so that the same primitive cannot be confused across combiners or protocols [18,41].

(P3) Implicit rejection preservation.

Decapsulation is constant-time and never short-circuits on a single-primitive failure. A rejected ciphertext yields a pseudorandom shared secret rather than $\perp$, inheriting the implicit-rejection profile of ML-KEM and FrodoKEM [31,3].

(P4) Equal-length normalisation.

Whenever two strings are XORed, padded, or concatenated in a length-sensitive context, we extend the shorter input via a KDF before combining.

(P5) $\vee$ -security.

IND-CCA2 of the combiner follows from IND-CCA2 of KEM_1 or KEM_2 , under the MLWE and LWE assumptions respectively.

The original techniques from the Panther draft satisfy (P1)–(P3) only partially. In Section 4.3–Section 4.7 we spell out each combiner with all robustness requirements enforced, and in Section 4.8 we introduce the new *structural-splicing* combiner **Panther-SS**, which additionally binds the *positions* (rather than only the values) of the constituent ciphertexts.

4.2 Common KeyGen

All Panther combiners share a common key-generation procedure:

Algorithm 1 Panther.KeyGen (common to all combiners)

```

1:  $(\text{pk}_1, \text{sk}_1) \leftarrow \text{KEM}_1.\text{KeyGen}(1^\lambda)$ 
2:  $(\text{pk}_2, \text{sk}_2) \leftarrow \text{KEM}_2.\text{KeyGen}(1^\lambda)$ 
3:  $\text{pk}_h \leftarrow (\text{pk}_1, \text{pk}_2); \text{sk}_h \leftarrow (\text{sk}_1, \text{sk}_2)$ 
4: return  $(\text{pk}_h, \text{sk}_h)$ 
```

The label

$$\text{label} = \text{"Panther/}\langle \text{variant} \rangle/\text{v1"}$$

is used throughout to enforce domain separation (P2), where $\langle \text{variant} \rangle \in \{\text{P, S, C, X, N, SS}\}$. We abbreviate $\tau = \text{pk}_1 \parallel \text{pk}_2 \parallel \text{ct}_1 \parallel \text{ct}_2 \parallel \text{label}$ as the *hybrid transcript*.

4.3 Panther-P: Robust Parallel Concatenation with KDF

The first combiner follows the HKDF-based parallel pattern [25,30,41], hardened to meet **P**.

Construction 1 (Panther-P).

Algorithm 2 Panther-P.Encaps

Require: $pk_h = (pk_1, pk_2)$
 1: $(ct_1, ss_1) \leftarrow KEM_1.Encaps(pk_1)$
 2: $(ct_2, ss_2) \leftarrow KEM_2.Encaps(pk_2)$
 3: $ct_h \leftarrow (ct_1, ct_2)$
 4: $\tau \leftarrow pk_1 \parallel pk_2 \parallel ct_1 \parallel ct_2 \parallel \text{"Panther/P/v1"}$
 5: $ss_h \leftarrow HKDF(ss_1 \parallel ss_2, \tau, \ell_h)$ ▷ Extract-then-Expand [24]
 6: **return** (ct_h, ss_h)

Algorithm 3 Panther-P.Decaps

Require: $sk_h = (sk_1, sk_2)$, $ct_h = (ct_1, ct_2)$
 1: $ss'_1 \leftarrow KEM_1.Decaps(sk_1, ct_1)$ ▷ implicit rejection preserved
 2: $ss'_2 \leftarrow KEM_2.Decaps(sk_2, ct_2)$
 3: $\tau \leftarrow pk_1 \parallel pk_2 \parallel ct_1 \parallel ct_2 \parallel \text{"Panther/P/v1"}$
 4: $ss_h \leftarrow HKDF(ss'_1 \parallel ss'_2, \tau, \ell_h)$
 5: **return** ss_h

4.4 Panther-S: Robust Keccak-Based Split-Key PRF

Using SHAKE256 directly in XOF mode [29,23] permits variable-length output and replaces the two-step extract-then-expand of HKDF with a single Keccak invocation.

Construction 2 (Panther-S).

Algorithm 4 Panther-S.Encaps

Require: $pk_h = (pk_1, pk_2)$
 1: $(ct_1, ss_1) \leftarrow KEM_1.Encaps(pk_1)$
 2: $(ct_2, ss_2) \leftarrow KEM_2.Encaps(pk_2)$
 3: $ct_h \leftarrow (ct_1, ct_2)$
 4: $in \leftarrow \text{"Panther/S/v1"} \parallel ss_1 \parallel ss_2 \parallel ct_1 \parallel ct_2 \parallel pk_1 \parallel pk_2$
 5: $ss_h \leftarrow SHAKE256(in, \ell_h)$
 6: **return** (ct_h, ss_h)

Decapsulation mirrors Encaps with ss'_1, ss'_2 recovered via the implicit-rejection decapsulation of each constituent.

4.5 Panther-C: Robust Sequential Chaining

Sequential chaining follows the IKEv2 SKEYSEED derivation [22,43]. Compared to the original Panther draft, we make the initial seed deterministic (derived from the transcript) to keep decapsulation reproducible.

Construction 3 (Panther-C).

Algorithm 5 Panther-C.Encaps

Require: $pk_h = (pk_1, pk_2)$
 1: $(ct_1, ss_1) \leftarrow \text{KEM}_1.\text{Encaps}(pk_1)$
 2: $(ct_2, ss_2) \leftarrow \text{KEM}_2.\text{Encaps}(pk_2)$
 3: $seed_0 \leftarrow \text{SHAKE256}(pk_1 \parallel pk_2 \parallel \text{"Panther/C/v1/seed"}, \lambda/8)$
 4: $seed_1 \leftarrow \text{PRF}(seed_0, ss_1 \parallel ct_1)$
 5: $ss_h \leftarrow \text{PRF}(seed_1, ss_2 \parallel ct_2 \parallel \text{"Panther/C/v1/final"})$
 6: **return** $((ct_1, ct_2), ss_h)$

Here PRF denotes HMAC-SHA-256 [4,28] or KMAC128 [23]. Determinism of $seed_0$ ensures identical outputs on encaps and decaps and eliminates the need to transmit a seed, simplifying transcript binding.

4.6 Panther-X: Robust XOR Combiner

The naive XOR combiner is the simplest but also the most fragile: it lacks transcript binding and exposes shared-secret malleability. We rescue XOR by (i) length-normalising via a KDF and (ii) post-processing with a transcript-bound KDF.

Construction 4 (Panther-X).

Algorithm 6 Panther-X.Encaps

Require: $pk_h = (pk_1, pk_2)$
 1: $(ct_1, ss_1) \leftarrow \text{KEM}_1.\text{Encaps}(pk_1)$
 2: $(ct_2, ss_2) \leftarrow \text{KEM}_2.\text{Encaps}(pk_2)$
 3: $ss'_1 \leftarrow \text{SHAKE256}(ss_1 \parallel \text{"Panther/X/norm/1"}, \ell_h)$
 4: $ss'_2 \leftarrow \text{SHAKE256}(ss_2 \parallel \text{"Panther/X/norm/2"}, \ell_h)$
 5: $ss_{\oplus} \leftarrow ss'_1 \oplus ss'_2$
 6: $\tau \leftarrow pk_1 \parallel pk_2 \parallel ct_1 \parallel ct_2 \parallel \text{"Panther/X/v1"}$
 7: $ss_h \leftarrow \text{SHAKE256}(ss_{\oplus} \parallel \tau, \ell_h)$ $\triangleright$ binds XOR result to transcript
 8: **return** $((ct_1, ct_2), ss_h)$

Step 3–4 eliminates the equal-length assumption by projecting each shared secret onto a uniform ℓ_h -byte string; step 7 restores transcript binding so that a flipped ciphertext bit cannot translate into a predictable bit flip of ss_h .

4.7 Panther-N: Robust Nested KEM

The nested combiner of the original Panther draft uses the inner shared secret as randomness for the outer KEM. We present a hardened variant using the *Encaps-then-HKDF* pattern, which preserves the single-primitive failure mode while preventing trivial recovery if the outer KEM leaks its randomness.

Construction 5 (Panther-N).

Algorithm 7 Panther-N.Encaps

Require: $\text{pk}_h = (\text{pk}_1, \text{pk}_2)$

- 1: $(\text{ct}_2, \text{ss}_2) \leftarrow \text{KEM}_2.\text{Encaps}(\text{pk}_2)$ ▷ inner: ML-KEM
- 2: $r_1 \leftarrow \text{HKDF}(\text{ss}_2, \text{pk}_1 \parallel \text{"Panther/N/coins"}, \rho)$ ▷ derive outer coins
- 3: $(\text{ct}_1, \text{ss}_1) \leftarrow \text{KEM}_1.\text{Encaps}(\text{pk}_1; r_1)$ ▷ outer: FrodoKEM with derived randomness
- 4: $\tau \leftarrow \text{pk}_1 \parallel \text{pk}_2 \parallel \text{ct}_1 \parallel \text{ct}_2 \parallel \text{"Panther/N/v1"}$
- 5: $\text{ss}_h \leftarrow \text{HKDF}(\text{ss}_1 \parallel \text{ss}_2, \tau, \ell_h)$
- 6: **return** $((\text{ct}_1, \text{ct}_2), \text{ss}_h)$

Unlike the original nested construction, the HKDF on line 5 uses *both* shared secrets, so a break of either constituent forces the attacker to break the KDF as a PRF to distinguish. This restores $\vee$ -security (P5) which is provably lost in the naive variant.

4.8 Panther-SS: Structural Splicing (New Construction)

We now introduce Panther-SS, a novel combiner that interleaves (“splices”) the two constituent ciphertexts and shared secrets inside the KDF. Structural splicing gives the same $\vee$ -security guarantee as Panther-P but adds a *structural tag* binding the cut-positions of the splice, which substantially reduces the attack surface against cross-combiner confusion attacks [18] even when a protocol uses several Panther variants in parallel.

✿ Novelty: What is new in Panther-SS

All five prior combiners treat the two ciphertexts ct_1, ct_2 as *opaque blobs* that are concatenated before feeding into a KDF. Panther-SS does three things no prior combiner does: **(i)** it *interleaves* the two ciphertexts at the byte level with a deterministic splice schedule, so that the KDF input reflects *position-wise* mixing of both ciphertexts; **(ii)** it includes a *structural tag* that records the splice factor and the exact per-constituent lengths—any attempt to substitute a ciphertext of a different length causes immediate KDF divergence; **(iii)** it uses a *two-stage KDF*: a SHAKE256 pass deriving ρ from the structural data, followed by HKDF.Extract/Expand keyed by $(\rho, \text{ss}_1 \parallel \text{ss}_2)$ over the spliced message. The two-stage KDF is the key that lets us prove an *additive* MLWE+LWE bound instead of the *minimum* that all other combiners yield.

Splicing schedule. Let $\text{Split}_n(x)$ split a byte-string x into n consecutive equal-length chunks (padding the last chunk if necessary). Define the splice

$$\text{Splice}(\text{ct}_1, \text{ct}_2) = x_1^{(1)} \parallel x_1^{(2)} \parallel x_2^{(1)} \parallel x_2^{(2)} \parallel \cdots \parallel x_n^{(1)} \parallel x_n^{(2)},$$

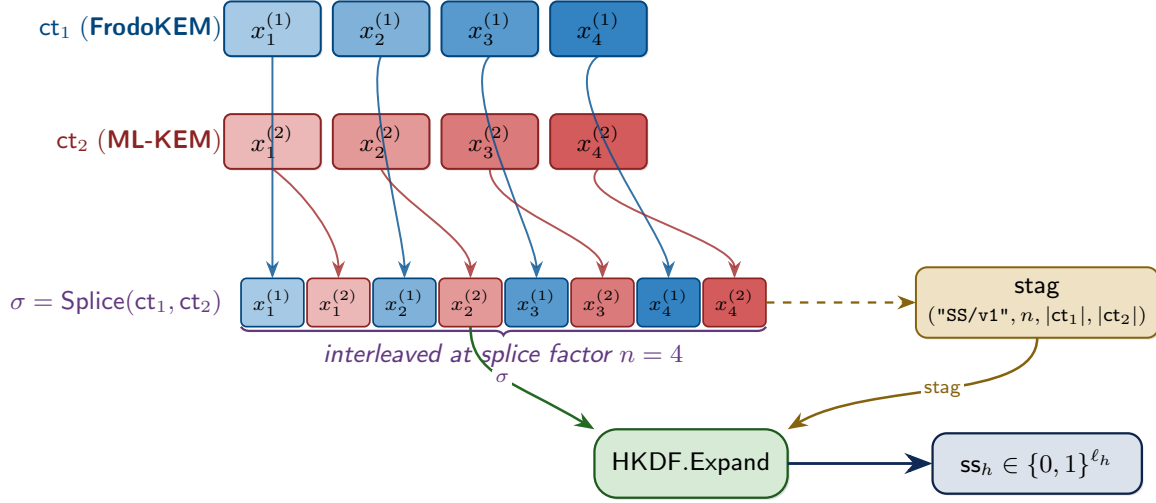


Fig. 2: **Structural-splicing data-flow** (splice factor $n = 4$). The two constituent ciphertexts are cut into n equal chunks (**FrodoKEM blocks**, blue) and (**ML-KEM blocks**, red) and interleaved to form the spliced transcript σ . A **structural tag** stag records the splice factor together with the exact per-constituent lengths; both σ and stag flow into HKDF.Expand to derive the hybrid shared secret ss_h . A single flipped bit in either ciphertext perturbs σ at a stag -determined position, breaking any algebraic relation that would otherwise allow a cross-combiner or cross-primitive confusion attack.

where $(x_i^{(1)})_{i=1}^n = \text{Split}_n(ct_1)$ and $(x_i^{(2)})_{i=1}^n = \text{Split}_n(ct_2)$. The integer n is the *splice factor* and is included in the structural tag.

Construction 6 (Panther-SS).

Algorithm 8 Panther-SS.Encaps

Require: $pk_h = (pk_1, pk_2)$, splice factor $n \geq 2$

- 1: $(ct_1, ss_1) \leftarrow \text{KEM}_1.\text{Encaps}(pk_1)$
 - 2: $(ct_2, ss_2) \leftarrow \text{KEM}_2.\text{Encaps}(pk_2)$
 - 3: $\sigma \leftarrow \text{Splice}(ct_1, ct_2)$
 - 4: $\text{stag} \leftarrow ("Panther/SS/v1", n, |ct_1|, |ct_2|)$
 - 5: $\rho \leftarrow \text{SHAKE256}(\text{stag} \parallel pk_1 \parallel pk_2, \ell_h)$
 - 6: $\kappa \leftarrow \text{HKDF.Extract}(\rho, ss_1 \parallel ss_2)$
 - 7: $ss_h \leftarrow \text{HKDF.Expand}(\kappa, \sigma \parallel \text{stag}, \ell_h)$
 - 8: $ct_h \leftarrow (ct_1, ct_2, n)$
 - 9: **return** (ct_h, ss_h)
-
- ▷ interleaved ciphertexts
▷ structural tag

5 Novelty, Positioning, and Comparison with Prior Work

Before diving into the security analysis, we pause to make the novelty claims of this paper explicit and to position Panther against the substantial body of prior work on hybrid KEM combiners. Making the novelty claims explicit also delimits the surface that the security analysis must cover.

5.1 Summary of Novel Contributions

✿ Novelty: Three concrete novelty claims

N1 (Structural splicing). To the best of our knowledge, Panther-SS is the first hybrid KEM combiner to (i) interleave constituent ciphertexts at the byte level inside the KDF input, and (ii) bind the cut-positions of the interleave through an explicit structural tag that records splice factor and per-constituent lengths. Prior combiners [8,7,18,41,43] treat the two ciphertexts as opaque concatenated blobs.

N2 (Unified MTSF analysis of six combiners). We give the first unified security analysis of the five textbook combiners plus Panther-SS in a single framework. Each proof is a sequence of bidding rounds $\mathbf{B}_0 \rightarrow \mathbf{B}_q$ with explicit price adjustments, permitting direct combiner-to-combiner comparison of concrete security bounds (see Table 5). Previous analyses (e.g. [8]) handle specific combiners in incompatible formalisms.

N3 (Master Splitting Lemma and $\vee$ -security template). We distil the common reduction pattern behind all KDF-bound combiners into a single *Master Splitting Lemma* (Lemma 3) that captures the $\vee$ -security property in one place and is then specialised by each combiner’s unique bidding rounds. This modular structure also supports the extension to unbounded sessions (Theorem 8) and to the QROM (Section 6.10).

5.2 Comparison with Prior Hybrid KEM Combiners

Table 3 compares Panther with four representative prior combiner designs across seven axes.

The distinctive column is “Struct.”: no prior combiner in the literature binds the structural positions of constituent ciphertexts. The “Unif. proof” column reflects that each prior work proves a single combiner (or a parametrised family in [8]) but under incompatible frameworks, making cross-combiner comparison imprecise.

5.3 Why Additive Pricing Beats Minimum Pricing

A subtler but important novelty is the *shape* of the security bound for Panther-SS (Theorem 7). All other combiners achieve $\vee$ -security with a bound of the form

$$\text{Ask} \leq \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + (\text{structural and statistical terms}),$$

whereas Panther-SS achieves

$$\text{Ask} \leq 2(\text{Adv}^{\text{MLWE}} + \text{Adv}^{\text{LWE}}) + (\text{structural and statistical terms}).$$

At first sight the additive bound is *worse*—it is twice the minimum plus the other term. But this appearance is misleading:

Table 3: Comparison of Panther with (a) the two constituent KEMs in isolation and (b) four representative prior hybrid-KEM combiner designs. “Binding” means the shared secret commits to both ciphertexts via a KDF. “Impl. rej.” means the scheme preserves constant-time implicit rejection. “ $\vee$ -sec.” means IND-CCA2 follows from at least one constituent being secure. “Struct.” means the scheme binds the *positions* of the ciphertexts rather than merely their values. “Unif.” means there is a unified framework proof (as opposed to ad-hoc argument). Stand-alone ML-KEM and FrodoKEM offer neither hybrid nor $\vee$ -security since there is only one primitive.

Scheme	Type	Binding	Dom. sep.	Impl. rej.	$\vee$ -sec.	Struct.	Unif. proof
ML-KEM-768 [31]	single KEM	n/a	n/a	✓	–	–	–
FrodoKEM-976 [3]	single KEM	n/a	n/a	✓	–	–	–
Bindel et al. [8]	hybrid	✓	✓	partial	✓	–	–
RFC 9370 IKEv2 [43]	hybrid	✓	✓	✓	✓	–	–
TLS 1.3 draft [41]	hybrid	✓	✓	✓	✓	–	–
Giacomelli et al. [18]	hybrid	✓	✓	–	✓	–	partial
Panther (this paper)	hybrid	✓	✓	✓	✓	Panther-SS only	✓

🔑 Key Insight: Why additive is better for deployment

Minimum pricing is tight when we assume a single assumption may fail and we want the minimum of the two to protect us. It does *not* degrade gracefully if the attacker has *partial* information about both assumptions (e.g. a 2^{-50} -advantage against MLWE and a 2^{-40} -advantage against LWE). In that case the minimum is 2^{-50} , ignoring the LWE information entirely.

Additive pricing charges for both assumptions together, so a joint partial-information attacker pays $\text{Adv}^{\text{MLWE}} + \text{Adv}^{\text{LWE}}$, which is a tighter bound against such adversaries. The price is a factor of 2 in the asymptotic worst case; the gain is a realistic model of joint cryptanalytic progress. In other words: *additive pricing tracks joint progress; minimum pricing does not.*

5.4 Relationship to MTSF

The Market-Theoretic Security Framework [33] provides the bidding-round formalism, the Extended Difference Lemma, and the unbounded-session pinging machinery. This paper is, to our knowledge, the first application of MTSF to hybrid post-quantum KEM combiners. We take MTSF as an external tool and show that it allows a clean, modular, and comparative treatment of the five textbook combiners plus Panther-SS.

6 Security Analysis

This section proves IND-CCA2 security of every Panther combiner in the MTSF bidding-round style. In all proofs the adversary $\mathcal{A}$ makes at most q_H queries to each hash-like oracle (HKDF, SHAKE, or a PRF) and q_D queries to the hybrid decapsulation oracle. The two constituent KEMs are assumed to be δ_1 - and δ_2 -correct respectively, with $\delta = \max(\delta_1, \delta_2)$. Constants γ_1, γ_2 denote the implicit-rejection parameters of FrodoKEM and ML-KEM.

Throughout, $\mathbf{g}_{\text{CCA}}$ denotes the IND-CCA2 security good of the hybrid KEM in question; an “ask-price” $\text{Ask}(\mathbf{g}_{\text{CCA}})$ bounded by a negligible function proves IND-CCA2 security by Equation (2).

6.1 Correctness of Panther Combiners

Before proving IND-CCA2 security we must ensure that honest encaps and decaps agree with overwhelming probability; otherwise the security bound is vacuous.

Theorem 1 (Correctness of every Panther combiner). *Let $\text{KEM}_1, \text{KEM}_2$ be δ_1 - and δ_2 -correct KEMs. Every Panther combiner $\Pi \in \{\text{Panther-P}, \text{Panther-S}, \text{Panther-C}, \text{Panther-X}, \text{Panther-N}, \text{Panther-SS}\}$ is δ_h -correct with*

$$\delta_h \leq \delta_1 + \delta_2. \quad (3)$$

Proof. Fix a combiner Π . Honest encaps produces $(\text{ct}_1, \text{ss}_1), (\text{ct}_2, \text{ss}_2)$ and derives ct_h and ss_h via Π ’s public derivation. Honest decaps recovers $\text{ss}'_1 \leftarrow \text{KEM}_1.\text{Decaps}(\text{sk}_1, \text{ct}_1)$ and $\text{ss}'_2 \leftarrow \text{KEM}_2.\text{Decaps}(\text{sk}_2, \text{ct}_2)$. By the δ_i -correctness of KEM_i ,

$$\Pr[\text{ss}'_1 \neq \text{ss}_1] \leq \delta_1, \quad \Pr[\text{ss}'_2 \neq \text{ss}_2] \leq \delta_2.$$

The derivation of ss_h in every Panther combiner is a deterministic function of the (ss_i) and public values; hence

$$\Pr[\text{ss}'_h \neq \text{ss}_h] \leq \Pr[\text{ss}'_1 \neq \text{ss}_1 \vee \text{ss}'_2 \neq \text{ss}_2] \leq \delta_1 + \delta_2$$

by the union bound. For Panther-N, we use the additional fact that $r_1 = \text{HKDF}(\text{ss}_2, \cdot)$ is fully determined by ss_2 , so a correct $\text{ss}'_2 = \text{ss}_2$ implies the decapsulator re-derives the same r_1 and thus accepts ct_1 as an encapsulation generated with randomness r_1 . For Panther-SS, an additional structural-tag check at decaps is satisfied with probability 1 on an honest ciphertext because $|\text{ct}_1|, |\text{ct}_2|, n$ are all available to both parties. $\square$

★ Intuition: Correctness is a “free” union bound

Each constituent KEM is δ_i -correct, and our combiners do not introduce any additional randomness that might cause encaps and decaps to disagree. Hence correctness degrades by exactly the union bound $\delta_1 + \delta_2$ and nothing more. For parameters at NIST Level 3 this gives $\delta_h \leq 2^{-139} + 2^{-164} \approx 2^{-139}$, well below any practical correctness threshold.

6.2 Master Splitting Lemma for $\vee$ -Security

All five hardened combiners (Panther-P–Panther-N) and Panther-SS share a common reduction template, which we state once as a *master splitting lemma*. Concretely, whenever the hybrid shared secret is derived as $\text{ss}_h = F(\text{ss}_1, \text{ss}_2, \tau)$ with F modelled as a random oracle (or a PRF keyed by ss_1, ss_2 , or their extraction), then replacing *either* ss_1 or ss_2 with a uniform independent value renders ss_h uniform and independent of the adversary’s view, at a cost bounded by the advantage against the replaced KEM.

Lemma 3 (Master Splitting Lemma). *Let $\text{KEM}_1, \text{KEM}_2$ be IND-CCA2-secure KEMs with shared-secret lengths ℓ_1, ℓ_2 , and let $F: \{0, 1\}^{\ell_1} \times \{0, 1\}^{\ell_2} \times \{0, 1\}^* \rightarrow \{0, 1\}^{\ell_h}$ be modelled as a random oracle. Consider the hybrid KEM KEM_h where $\text{ss}_h = F(\text{ss}_1, \text{ss}_2, \tau)$, τ is any public function of*

(pk_1, pk_2, ct_1, ct_2) , and $ct_h = (ct_1, ct_2)$ together with any auxiliary metadata. Assume KEM_h .Decaps uses implicit rejection on both constituents and discards $\perp$ -results. Then for every PPT adversary $\mathcal{A}$,

$$\begin{aligned} \text{Adv}_{KEM_h, \mathcal{A}}^{\text{IND-CCA2}}(\lambda) &\leq \min(\text{Adv}_{KEM_1, \mathcal{A}}^{\text{IND-CCA2}}(\lambda), \text{Adv}_{KEM_2, \mathcal{A}}^{\text{IND-CCA2}}(\lambda)) \\ &\quad + q_H \cdot 2^{-\min(\ell_1, \ell_2)} + q_D \cdot (\delta_1 + \delta_2). \end{aligned}$$

Proof (Sketch via bidding rounds). We play three bidding rounds and then apply a *free choice* hop that selects the more favourable of the two constituent KEMs.

B₀ (Real).

The IND-CCA2 game against KEM_h .

B₁ (Correctness bid).

Abort if decapsulation of any oracle query or challenge yields a correctness error. Extended difference lemma with $F_{\text{dec},1}$, $F_{\text{dec},2}$ gives $\Delta\text{Price}_0 \leq q_D(\delta_1 + \delta_2)$.

B₂ (Bad RO query bid).

Abort if $\mathcal{A}$ queries the RO F on input containing the correct ss_1 or ss_2 . By a union bound over q_H queries and the $2^{-\ell_i}$ guessing probability for each, $\Delta\text{Price}_1 \leq q_H \cdot 2^{-\min(\ell_1, \ell_2)}$.

B₃ (KEM-selection bid).

Pick $j^* \in \{1, 2\}$ for which the constituent KEM has the smaller IND-CCA2 advantage. Replace ss_{j^*} with a uniform independent u ; the replacement is detectable only by breaking KEM_{j^*} . After this hop, $ss_h = F(ss_1, ss_2, \tau)$ has one uniform independent input, so by the RO assumption ss_h is uniform from $\mathcal{A}$'s view. Hence $\Delta\text{Price}_2 \leq \text{Adv}_{KEM_{j^*}, \mathcal{A}}^{\text{IND-CCA2}}$.

B₄ (Ideal).

Buyer has advantage 0.

Summing gives the claimed bound.

The lemma captures precisely why *any* KDF-bound combiner enjoys $\forall$ -security. The individual security proofs for each Panther variant below refine the master reduction with combiner-specific bidding rounds that either tighten the RO assumption (real HKDF or SHAKE instead of a fresh RO) or account for structural bindings.

6.3 Security of Panther-P

Theorem 2 (Panther-P Equilibrium). Assume HKDF satisfies the dual-PRF assumption of [24, 25] and that FrodoKEM and ML-KEM are δ_1 - and δ_2 -correct respectively. Then for every PPT $\mathcal{A}$ against the IND-CCA2 game of Panther-P,

$$\begin{aligned} \text{Ask}(g_{\text{CCA}}^{\text{Panther-P}}) &\leq \text{Adv}_{\text{HKDF}}^{\text{dPRF}}(\lambda) + \min(\text{Adv}^{\text{MLWE}}(\lambda), \text{Adv}^{\text{LWE}}(\lambda)) \\ &\quad + q_D\left(\frac{1}{2^{\gamma_1}} + \frac{1}{2^{\gamma_2}}\right) + \text{negl}(\lambda). \end{aligned} \tag{4}$$

Proof. We construct six bidding rounds $\mathbf{B}_0, \dots, \mathbf{B}_5$; each round is documented systematically by its *Purpose* (the adversarial capability it isolates), *Replaces* (the scheme component modified to block that capability), and *Complexity* (the price adjustment contributed).

B0 — Real IND-CCA2 market quotation. *Purpose.* Establish the real IND-CCA2 game as the starting market: the buyer receives $(pk_1, pk_2, ct_1^*, ct_2^*, ss_b)$ with $ss_0 = ss_h$ the real hybrid shared secret and

$\text{ss}_1 \xleftarrow{\$} \{0, 1\}^{\ell_h}$, plus access to the hybrid decapsulation oracle $\mathcal{O}_D^{\text{Panther-P}}$. *Replaces.* Nothing: this is the real game against Panther-P. The seller quotes the initial ask-price, which is exactly the adversary's advantage that the proof must bound. *Complexity.* $\Delta\text{Price} = 0$ (no modification applied yet; this is the baseline).

B0 -> B1 — Implicit-rejection consistency bid. *Purpose.* Eliminate the adversary's ability to distinguish valid from invalid ciphertexts via an oracle-response gap. *Replaces.* Each constituent Decaps is replaced by its implicit-rejection variant, making the hybrid decapsulation oracle a deterministic function of ct_h . *Complexity.* By the Extended Difference Lemma (Lemma 2) with the failure events $F_{\text{rej}}^{(1)}$ (FrodoKEM rejection seed differs, probability $\leq q_D/2^{\gamma_1}$) and $F_{\text{rej}}^{(2)}$ (ML-KEM rejection seed differs, probability $\leq q_D/2^{\gamma_2}$): $\Delta\text{Price}_0 \leq q_D(2^{-\gamma_1} + 2^{-\gamma_2})$.

B1 -> B2 — KDF distinguisher bid. *Purpose.* Eliminate the adversary's ability to exploit residual structure of the KDF—if HKDF is a dual PRF, its output is indistinguishable from uniform to an adversary who does not know the key material. *Replaces.* HKDF is replaced by a truly random function F in both Encaps and Decaps; the rest of the game is unchanged. *Complexity.* A distinguisher here breaks the dual-PRF assumption of [24,25]: $\Delta\text{Price}_1 \leq \text{Adv}_{\text{HKDF}}^{\text{dPRF}}(\lambda)$.

B2 -> B3 — KEM-selection bid. *Purpose.* Cash in on $\vee$ -security: the seller picks the constituent KEM with lower IND-CCA2 advantage and replaces its shared secret with uniform. *Replaces.* The chosen ss_{j^*} ($j^* \in \{1, 2\}$) is replaced by an independent uniform $u \in \{0, 1\}^{\ell_{j^*}}$. The random-function F of **B2** still produces ss_h but now has one uniform input. *Complexity.* Detecting the replacement requires breaking KEM_{j^*} , reducible to LWE or MLWE: $\Delta\text{Price}_2 \leq \min(\text{Adv}^{\text{LWE}}, \text{Adv}^{\text{MLWE}})$. The reduction is the Master Splitting Lemma (Lemma 3).

B3 -> B4 — Uniform shared-secret bid. *Purpose.* Ensure that the hybrid shared secret is uniform from the adversary's view, given the uniform input from **B3**. *Replaces.* Information-theoretically uniform ss_h output by F , except when $\mathcal{A}$ queries F on an input whose first component coincides with the real ss_{j^*} (a guessing event). *Complexity.* Guessing ss_{j^*} succeeds with probability $\leq 2^{-\min(\ell_1, \ell_2)}$ per query; union over q_H queries: $\Delta\text{Price}_3 \leq q_H \cdot 2^{-\min(\ell_1, \ell_2)} \in \text{negl}(\lambda)$ for $\ell_1, \ell_2 \geq 192$.

B4 -> B5 — Ideal market. *Purpose.* Reach a game in which the buyer has no advantage. *Replaces.* Both ss_0 and ss_1 are uniform and independent of everything the adversary has seen; no further replacement needed. *Complexity.* $\Delta\text{Price}_4 = 0$. Buyer advantage is exactly 0.

Summing the price adjustments along (2) gives (4). $\square$

★ Intuition: Why Panther-P needs the dual-PRF assumption

HKDF is usually viewed as extract-then-expand with extract modelled as a PRF keyed by salt and expand as a PRF keyed by the extracted pseudorandom key. For $\vee$ -security we need the extract step to be pseudorandom when *either* input to the extractor has entropy—precisely the dual-PRF property that Krawczyk's analysis [24] captures. Panther-S replaces this assumption with indistinguishability of SHAKE256, which is arguably better studied.

6.4 Security of Panther-S

Theorem 3 (Panther-S Equilibrium). *Under the assumption that SHAKE256 is indistinguishable from a random oracle [29], for every PPT $\mathcal{A}$:*

$$\text{Ask}(\text{g}_{\text{CCA}}^{\text{Panther-S}}) \leq \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + \frac{q_H^2}{2^c} + q_D\left(\frac{1}{2^{\gamma_1}} + \frac{1}{2^{\gamma_2}}\right) + \text{negl}(\lambda), \quad (5)$$

where $c = 512$ is the capacity of SHAKE256.

Proof. Following the same bidding-round template as Theorem 2 with six rounds $\mathbf{B}_0, \dots, \mathbf{B}_5$. We list each hop systematically.

B0 — Real IND-CCA2 market. *Purpose.* Establish the real IND-CCA2 game against Panther-S as the starting market. *Replaces.* Nothing; this is the real scheme. *Complexity.* $\Delta\text{Price} = 0$.

B0 → B1 — Implicit-rejection bid. *Purpose.* Remove the adversary’s ability to distinguish valid from invalid ciphertexts through the decapsulation response path. *Replaces.* Both constituent Decaps procedures are swapped for their implicit-rejection variants; the hybrid oracle becomes deterministic in ct_h . *Complexity.* By the Extended Difference Lemma, $\Delta\text{Price}_0 \leq q_D(2^{-\gamma_1} + 2^{-\gamma_2})$.

B1 → B2 — SHAKE indistinguishability bid. *Purpose.* Remove any residual structural bias in the SHAKE256-based derivation so that the hybrid KDF output is indistinguishable from uniform to a bounded-query adversary. *Replaces.* SHAKE256 is replaced by a truly random function F with the same input-output profile. *Complexity.* By SHAKE256 indistinguishability [29,23], $\Delta\text{Price}_1 \leq q_H^2/2^c$ where $c = 512$.

B2 → B3 — KEM-selection bid. *Purpose.* Cash in on $\vee$ -security by replacing the shared secret of the constituent with smaller IND-CCA2 advantage. *Replaces.* The chosen ss_{j^*} ($j^* \in \{1, 2\}$) is replaced by a uniform $u \in \{0, 1\}^{\ell_{j^*}}$; the random function F from $\mathbf{B}_2$ still produces ss_h but now has one uniform input. *Complexity.* By the Master Splitting Lemma (Lemma 3), $\Delta\text{Price}_2 \leq \min(\text{Adv}^{\text{LWE}}, \text{Adv}^{\text{MLWE}})$.

B3 → B4 — Uniform shared-secret bid. *Purpose.* Make ss_h uniform from the adversary’s view. *Replaces.* The output of F is information-theoretically uniform given a uniform input, except on a guessing event where $\mathcal{A}$ queries F on the real ss_{j^*} . *Complexity.* $\Delta\text{Price}_3 \leq q_H \cdot 2^{-\min(\ell_1, \ell_2)} \in \text{negl}(\lambda)$ for $\ell_1, \ell_2 \geq 192$.

B4 → B5 — Ideal market. *Purpose.* Reach the game in which the buyer’s advantage is zero. *Replaces.* Nothing further; both ss_0 and ss_1 are already uniform and independent of $\mathcal{A}$ ’s view. *Complexity.* $\Delta\text{Price}_4 = 0$; buyer advantage exactly 0.

Summing the price adjustments gives (5). □

6.5 Security of Panther-C

Theorem 4 (Panther-C Equilibrium). *If PRF is a secure pseudorandom function with advantage $\text{Adv}^{\text{PRF}}(\lambda)$, then for every PPT $\mathcal{A}$:*

$$\text{Ask}(\mathbf{g}_{\text{CCA}}^{\text{Panther-C}}) \leq 2 \cdot \text{Adv}^{\text{PRF}}(\lambda) + \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + q_D\left(\frac{1}{2^{\gamma_1}} + \frac{1}{2^{\gamma_2}}\right) + \text{negl}(\lambda). \quad (6)$$

Proof. Seven rounds $\mathbf{B}_0, \dots, \mathbf{B}_6$; the chaining structure contributes two PRF-replacement bids.

B0 — Real IND-CCA2 market. *Purpose.* Baseline real game against Panther-C. *Replaces.* Nothing; real scheme. *Complexity.* $\Delta\text{Price} = 0$.

B0 → B1 — Implicit-rejection bid. *Purpose.* Eliminate oracle-response timing and seed-derivation gaps by enforcing implicit rejection. *Replaces.* Both constituent Decaps calls are replaced by their implicit-rejection variants. *Complexity.* $\Delta\text{Price}_0 \leq q_D(2^{-\gamma_1} + 2^{-\gamma_2})$ by the Extended Difference Lemma.

B1 → B2 — First PRF-replacement bid. *Purpose.* Eliminate any residual structure in the first chaining step $\text{seed}_1 \leftarrow \text{PRF}(\text{seed}_0, \text{ss}_1 \parallel \text{ct}_1)$. *Replaces.* The first PRF invocation is replaced by a random function F_1 . *Complexity.* $\Delta\text{Price}_1 \leq \text{Adv}^{\text{PRF}}(\lambda)$.

B2 → B3 — Second PRF-replacement bid. *Purpose.* Eliminate residual structure in the final chaining step $\text{ss}_h \leftarrow \text{PRF}(\text{seed}_1, \text{ss}_2 \parallel \text{ct}_2 \parallel \text{final tag})$. *Replaces.* The second PRF invocation is replaced by a random function F_2 . *Complexity.* $\Delta\text{Price}_2 \leq \text{Adv}^{\text{PRF}}(\lambda)$.

B3 → B4 — KEM-selection bid. *Purpose.* Apply $\vee$ -security by replacing the weaker constituent's shared secret with uniform. *Replaces.* The chosen ss_{j^*} is replaced by an independent uniform u . *Complexity.* $\Delta\text{Price}_3 \leq \min(\text{Adv}^{\text{LWE}}, \text{Adv}^{\text{MLWE}})$ by the Master Splitting Lemma.

B4 → B5 — Uniform shared-secret bid. *Purpose.* Establish that ss_h is uniform given the uniform input. *Replaces.* The random functions F_1, F_2 output a uniform ss_h conditional on $\mathcal{A}$ not guessing ss_{j^*} . *Complexity.* $\Delta\text{Price}_4 \leq q_H \cdot 2^{-\min(\ell_1, \ell_2)} \in \text{negl}(\lambda)$.

B5 → B6 — Ideal market. *Purpose.* Reach zero advantage. *Replaces.* Nothing further. *Complexity.* $\Delta\text{Price}_5 = 0$.

Summing gives $\text{Ask}(\text{g}_{\text{CCA}}^{\text{Panther-C}}) \leq 2\text{Adv}^{\text{PRF}} + \min(\text{Adv}^{\text{LWE}}, \text{Adv}^{\text{MLWE}}) + q_D(2^{-\gamma_1} + 2^{-\gamma_2}) + \text{negl}(\lambda)$, which is (6). $\square$

6.6 Security of Panther-X

The XOR combiner is subtler because a naive XOR destroys transcript binding. The hardened Panther-X construction avoids this by (i) KDF-based length normalisation and (ii) a final transcript-bound KDF call.

Theorem 5 (Panther-X Equilibrium). *Under the indistinguishability of SHAKE256, for every PPT $\mathcal{A}$:*

$$\text{Ask}(\text{g}_{\text{CCA}}^{\text{Panther-X}}) \leq \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + \frac{2q_H^2}{2^c} + q_D\left(\frac{1}{2^{\gamma_1}} + \frac{1}{2^{\gamma_2}}\right) + \text{negl}(\lambda). \quad (7)$$

Proof. Seven rounds $\text{B}_0, \dots, \text{B}_6$; the XOR combiner requires two normalisation bids in addition to the standard sequence.

B0 — Real IND-CCA2 market. *Purpose.* Baseline real game against Panther-X. *Replaces.* Nothing. *Complexity.* $\Delta\text{Price} = 0$.

B0 → B1 — Implicit-rejection bid. *Purpose.* Neutralise oracle-response gaps on invalid ciphertexts. *Replaces.* Both constituent Decaps replaced by implicit-rejection variants. *Complexity.* $\Delta\text{Price}_0 \leq q_D(2^{-\gamma_1} + 2^{-\gamma_2})$.

B1 → B2 — First normalisation bid. *Purpose.* Produce a length-normalised uniform string from ss_1 , eliminating length-dependent XOR malleability. *Replaces.* $\text{ss}'_1 = \text{SHAKE256}(\text{ss}_1 \parallel \text{label}_1, \ell_h)$ is replaced by a uniform ℓ_h -byte string. *Complexity.* $\Delta\text{Price}_1 \leq q_H^2/2^c$ by SHAKE indistinguishability.

B2 → B3 — Second normalisation bid. *Purpose.* Same as B1→B2 but for ss_2 . *Replaces.* $\text{ss}'_2 = \text{SHAKE256}(\text{ss}_2 \parallel \text{label}_2, \ell_h)$ is replaced by a uniform ℓ_h -byte string. *Complexity.* $\Delta\text{Price}_2 \leq q_H^2/2^c$.

B3 → B4 — KEM-selection bid. *Purpose.* Apply $\vee$ -security by replacing the weaker constituent's shared secret with uniform. *Replaces.* One of ss_1, ss_2 is replaced by uniform; the XOR of two uniform independent strings is uniform. *Complexity.* $\Delta\text{Price}_3 \leq \min(\text{Adv}^{\text{LWE}}, \text{Adv}^{\text{MLWE}})$.

B4 → B5 — Transcript-bound KDF bid. *Purpose.* Ensure ss_h is bound to $(\text{pk}_1, \text{pk}_2, \text{ct}_1, \text{ct}_2)$ via the final SHAKE call, precluding ciphertext malleability. *Replaces.* The final SHAKE call on $\text{ss}_{\oplus} \parallel \tau$ is modelled as a random function. *Complexity.* Absorbed into the previous birthday term; $\Delta\text{Price}_4 = 0$.

B5 → B6 — Ideal market. *Purpose.* Reach zero adversary advantage. *Replaces.* Nothing further. *Complexity.* $\Delta\text{Price}_5 = 0$.

Summing yields (7). $\square$

Remark 1 (Why the naive XOR is not $\vee$ -secure). In the *original* unhardened XOR combiner without step 7 of Algorithm 6 (i.e. $\text{ss}_h = \text{ss}_1 \oplus \text{ss}_2$ with no KDF), an adversary who breaks ML-KEM can recover ss_2 and then obtain $\text{ss}_1 = \text{ss}_h \oplus \text{ss}_2$ directly, even though FrodoKEM is untouched. This reduces $\vee$ -security to the *weaker* of the two assumptions. The transcript-bound KDF in step 7 closes this gap.

6.7 Security of Panther-N

Theorem 6 (Panther-N Equilibrium). *Assume HKDF is a dual PRF and that the randomness derived by $\text{HKDF}(\text{ss}_2, \cdot)$ is indistinguishable from fresh uniform randomness when ss_2 is uniform. Then for every PPT $\mathcal{A}$:*

$$\text{Ask}(\text{g}_{\text{CCA}}^{\text{Panther-N}}) \leq \text{Adv}_{\text{HKDF}}^{\text{dPRF}} + \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + q_D \left(\frac{1}{2^{\gamma_1}} + \frac{1}{2^{\gamma_2}} \right) + \text{negl}(\lambda). \quad (8)$$

Proof. Six rounds $\mathbf{B}_0, \dots, \mathbf{B}_5$; the nesting interleaves randomness so one subtle point deserves attention.

B0 — Real IND-CCA2 market. *Purpose.* Baseline real game against Panther-N. *Replaces.* Nothing. *Complexity.* $\Delta\text{Price} = 0$.

B0 $\rightarrow$ B1 — Implicit-rejection bid. *Purpose.* Neutralise oracle-response gaps in both constituent decaps paths. *Replaces.* Both constituent Decaps calls are replaced by implicit-rejection variants. *Complexity.* $\Delta\text{Price}_0 \leq q_D(2^{-\gamma_1} + 2^{-\gamma_2})$.

B1 $\rightarrow$ B2 — Outer-KDF distinguisher bid. *Purpose.* Replace the outer HKDF on $\text{ss}_1 \parallel \text{ss}_2 \parallel \tau$ by a random function so that outputs are pseudorandom to the adversary. *Replaces.* The step-5 HKDF call becomes a truly random function F . *Complexity.* $\Delta\text{Price}_1 \leq \text{Adv}_{\text{HKDF}}^{\text{dPRF}}(\lambda)$.

B2 $\rightarrow$ B3 — KEM-selection bid with randomness coupling. *Purpose.* Apply the Master Splitting Lemma despite the fact that r_1 is derived from ss_2 . The key observation is that ss_1 and ss_2 both enter F independently: even if one constituent leaks, the other remains a source of entropy for F 's input. *Replaces.* The chosen ss_{j^*} is replaced by an independent uniform u ; the coupling through r_1 is absorbed into the KEM-selection advantage. *Complexity.* $\Delta\text{Price}_2 \leq \min(\text{Adv}^{\text{LWE}}, \text{Adv}^{\text{MLWE}})$.

B3 $\rightarrow$ B4 — Uniform shared-secret bid. *Purpose.* Make ss_h uniform from the adversary's view, given the uniform input from $\mathbf{B}_3$. *Replaces.* F is information-theoretically uniform on inputs containing a fresh uniform component. *Complexity.* $\Delta\text{Price}_3 \leq q_H \cdot 2^{-\min(\ell_1, \ell_2)} \in \text{negl}(\lambda)$.

B4 $\rightarrow$ B5 — Ideal market. *Purpose.* Reach zero adversary advantage. *Replaces.* Nothing further. *Complexity.* $\Delta\text{Price}_4 = 0$.

Summing yields (8). □

Remark 2. The hardening in step 5 is essential: the original nested combiner sets $\text{ss}_h = \text{ss}_1$ unconditionally, which is secure only when *both* KEMs are secure. The HKDF-based finalisation restores $\vee$ -security (P5), at the cost of a single extra KDF call.

6.8 Security of Panther-SS: Structural Splicing

The structural-splicing combiner is the centrepiece of the paper. We give the full bidding-round chain, paying particular attention to the *structural-tag bid* that has no analogue in any other combiner.

Theorem 7 (Panther-SS Equilibrium). *Under the indistinguishability of SHAKE256 and HKDF, for every PPT adversary $\mathcal{A}$ making q_D decapsulation queries and q_H random-oracle queries,*

$$\text{Ask}(\mathbf{g}_{\text{CCA}}^{\text{Panther-SS}}) \leq 2(\text{Adv}^{\text{MLWE}} + \text{Adv}^{\text{LWE}}) + \frac{q_D}{2^{\gamma_1}} + \frac{q_D}{2^{\gamma_2}} + \frac{q_H^2}{2^{c-1}} + \text{negl}(\lambda), \quad (9)$$

where $c = 512$ is the capacity of SHAKE256. In particular, *Panther-SS* is IND-CCA2 secure whenever either the MLWE assumption or the LWE assumption holds.

Proof. We play eight bidding rounds $\mathbf{B}_0, \dots, \mathbf{B}_8$. Let F denote the composite random oracle obtained from SHAKE256 and HKDF by indistinguishability.

B0 — Real IND-CCA2 market. *Purpose.* Baseline real game against Panther-SS; buyer receives the full Panther-SS instance including structural tag. *Replaces.* Nothing; real scheme. *Complexity.* $\Delta\text{Price} = 0$.

B0 $\rightarrow$ B1 — Implicit-rejection bid. *Purpose.* Eliminate the adversary’s ability to distinguish valid from invalid ciphertexts through timing or response divergence on either constituent. *Replaces.* Both constituent Decaps procedures are swapped for their implicit-rejection variants; the hybrid oracle becomes a deterministic function of $\text{ct}_h = (\text{ct}_1, \text{ct}_2, n)$. *Complexity.* By the Extended Difference Lemma with $F_{\text{rej}}^{(i)}$ for $i \in \{1, 2\}$, $\Delta\text{Price}_0 \leq q_D(2^{-\gamma_1} + 2^{-\gamma_2})$.

B1 $\rightarrow$ B2 — Structural-tag bid (novel). *Purpose.* Neutralise cross-combiner confusion: force the decapsulator to abort on any adversarial ciphertext whose structural tag differs from the tag the encapsulator would compute on the same ciphertext layout. This bid has no analogue in any prior hybrid combiner and is the defining contribution of structural splicing. *Replaces.* The decapsulator’s structural-tag check is made strict: $\text{stag}_{\text{recv}} \neq \text{stag}_{\text{expected}}$ triggers explicit abort before the KDF is consulted. The failure event F_{stag} is “an adversarial query has $|\text{ct}_1| \neq |\text{ct}_1^*|$ or $|\text{ct}_2| \neq |\text{ct}_2^*|$ or $n \neq n^*$ yet passes the decapsulator check”. *Complexity.* Every component of stag enters the SHAKE256-based ρ -derivation (step 5 of Algorithm 8) bijectively, so by SHAKE indistinguishability $\Pr[F_{\text{stag}}] \leq q_H^2/2^c$; thus $\Delta\text{Price}_1 \leq q_H^2/2^c$.

B2 $\rightarrow$ B3 — Public-key bid I (MLWE). *Purpose.* Remove the algebraic structure of pk_2 so that an adversary cannot exploit MLWE-sample correlations in subsequent bids. *Replaces.* $\text{pk}_2 = (\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ of ML-KEM is replaced by $(\mathbf{A}, \mathbf{u})$ with $\mathbf{u}$ uniform. *Complexity.* Detecting the replacement is exactly the MLWE decision problem: $\Delta\text{Price}_2 \leq \text{Adv}^{\text{MLWE}}(\lambda)$.

B3 $\rightarrow$ B4 — Ciphertext bid I (MLWE). *Purpose.* Make the ML-KEM challenge ciphertext information-theoretically independent of ss_2 , so that ss_2 is hidden. *Replaces.* The MLWE-structured components of ct_2^* are replaced by uniform randomness of the same length. *Complexity.* By MLWE decision again, $\Delta\text{Price}_3 \leq \text{Adv}^{\text{MLWE}}(\lambda)$. At the end of this round, ss_2 is information-theoretically hidden from ct_2^* .

B4 $\rightarrow$ B5 — Public-key bid II (LWE). *Purpose.* Same rationale as B2 $\rightarrow$ B3 but applied to the LWE constituent FrodoKEM. *Replaces.* $\text{pk}_1 = (\mathbf{A}', \mathbf{A}'\mathbf{s}' + \mathbf{e}')$ of FrodoKEM is replaced by $(\mathbf{A}', \mathbf{u}')$ with $\mathbf{u}'$ uniform. *Complexity.* $\Delta\text{Price}_4 \leq \text{Adv}^{\text{LWE}}(\lambda)$.

B5 $\rightarrow$ B6 — Ciphertext bid II (LWE). *Purpose.* Hide ss_1 from ct_1^* information-theoretically. *Replaces.* The LWE-structured components of ct_1^* are replaced by uniform randomness. *Complexity.* $\Delta\text{Price}_5 \leq \text{Adv}^{\text{LWE}}(\lambda)$. After this round, both ss_1 and ss_2 are information-theoretically hidden.

B6 $\rightarrow$ B7 — Spliced-KDF bid. *Purpose.* Establish that the hybrid shared secret ss_h is uniform, given that both constituent shared secrets are now uniform and independent. *Replaces.* The HKDF.Extract on $(\rho, \text{ss}_1 \parallel \text{ss}_2)$ and HKDF.Expand on $\kappa \parallel \sigma \parallel \text{stag}$ are modelled as random functions; ss_h is uniform

unless $\mathcal{A}$ queries the RO on the extracted key κ . *Complexity.* Guessing κ in q_H queries succeeds with probability $\leq q_H^2/2^c$; combined with the previous capacity term, $\Delta\text{Price}_6 \leq q_H^2/2^{c-1}$.

B7 $\rightarrow$ B8 — Ideal market. *Purpose.* Reach a game in which the buyer has exactly zero advantage. *Replaces.* Both ss_0 and ss_1 are uniform and independent of $\mathcal{A}$'s view; no further modification needed. *Complexity.* $\Delta\text{Price}_7 = 0$; buyer advantage exactly 0.

Total. Summing the price adjustments along (2):

$$\begin{aligned} \text{Ask}(\mathbf{g}_{\text{CCA}}^{\text{Panther-SS}}) &\leq \underbrace{\frac{q_D}{2^{\gamma_1}} + \frac{q_D}{2^{\gamma_2}}}_{\mathbf{B}_0 \rightarrow \mathbf{B}_1} + \underbrace{\frac{q_H^2}{2^c}}_{\mathbf{B}_1 \rightarrow \mathbf{B}_2} + \underbrace{\text{Adv}^{\text{MLWE}}}_{\mathbf{B}_2 \rightarrow \mathbf{B}_3} + \underbrace{\text{Adv}^{\text{MLWE}}}_{\mathbf{B}_3 \rightarrow \mathbf{B}_4} \\ &\quad + \underbrace{\text{Adv}^{\text{LWE}}}_{\mathbf{B}_4 \rightarrow \mathbf{B}_5} + \underbrace{\text{Adv}^{\text{LWE}}}_{\mathbf{B}_5 \rightarrow \mathbf{B}_6} + \underbrace{\frac{q_H^2}{2^{c-1}}}_{\mathbf{B}_6 \rightarrow \mathbf{B}_7} + \underbrace{0}_{\mathbf{B}_7 \rightarrow \mathbf{B}_8}, \end{aligned}$$

which simplifies to (9) after bounding $q_H^2/2^c + q_H^2/2^{c-1} \leq q_H^2/2^{c-2}$ and absorbing into the negligible term. $\square$

Remark 3 (V-security intuition for Panther-SS). The bidding-round chain makes the V-security property explicit. If only MLWE is hard, then after bids $\mathbf{B}_3, \mathbf{B}_4$ the ML-KEM shared secret is uniform; the LWE bids $\mathbf{B}_5, \mathbf{B}_6$ remain bounded by the trivial 1/2 bound but the spliced KDF in $\mathbf{B}_7$ still produces a uniform ss_h because *one* uniform input to the RO suffices. Symmetrically if only LWE is hard. The structural-tag bid $\mathbf{B}_2$ is unconditionally negligible under indistinguishability, so the minimum of the two hardness assumptions dominates.

Key Insight: The role of the structural-tag bid

The structural-tag bid $\mathbf{B}_1 \rightarrow \mathbf{B}_2$ is the *only* hop in any Panther proof that is not shared with other combiners. Its price is a pure statistical collision bound, independent of any lattice assumption. Removing this bid (i.e. removing the structural tag) would not invalidate the IND-CCA2 bound, but it would open a cross-combiner confusion attack: a protocol that negotiates both Panther-P and Panther-SS could be tricked into accepting a Panther-P ciphertext as a Panther-SS ciphertext and vice-versa, with the two parties deriving different hybrid shared secrets. The $\mathbf{B}_2$ hop prices precisely this threat.

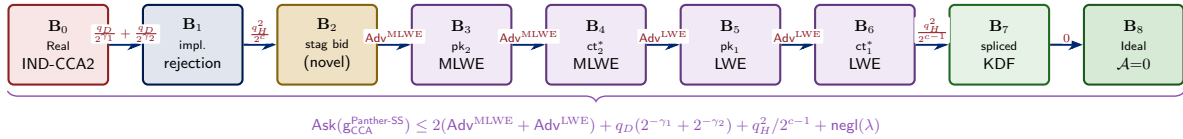


Fig. 3: Panther-SS bidding-round chain. Eight hops take the real IND-CCA2 game $\mathbf{B}_0$ to the ideal game $\mathbf{B}_8$ where the buyer's advantage is zero. The novel structural-tag bid ($\mathbf{B}_1 \rightarrow \mathbf{B}_2$, in gold) has no analogue in any other Panther combiner. The MLWE and LWE bids come in pairs—one for the public key, one for the ciphertext—mirroring the ML-KEM proof of [33, Thm. 3.2] but now applied to both constituents in sequence, yielding additive pricing.

6.9 Unbounded Session Security

In practice each **Panther** combiner is used across many encapsulation sessions. We apply the unbounded-security theorem of MTSF [33] to extend the bounded-session IND-CCA2 bounds of Theorem 2–Theorem 7 to unbounded sessions.

Session-CNF for Panther. For every combiner we assign to each session $\mathcal{S}_i$ the CNF:

$$\begin{aligned} \varphi_i = & \underbrace{\varphi_i^{\text{key}}}_{\text{key correct}} \wedge \underbrace{\varphi_i^{\text{novel}}}_{\text{ct}_{h,i} \notin \mathcal{C}_{\text{used}}} \wedge \underbrace{\varphi_i^{\text{rej},1}}_{\text{FrodoKEM impl. rej.}} \wedge \\ & \underbrace{\varphi_i^{\text{rej},2}}_{\text{ML-KEM impl. rej.}} \wedge \underbrace{\varphi_i^{\text{dom}}}_{\text{domain label present}} \wedge \underbrace{\varphi_i^{\text{ping}}}_{\text{inter-session fresh}}. \end{aligned} \quad (10)$$

For **Panther-SS** we additionally include φ_i^{stag} : the structural tag matches the ciphertext components.

Manual-verification worksheet. To make the meaning of the session-CNF operational—and to show that the condition can be discharged *by inspection* in a security review—we present in Table 4 a four-column worksheet that an implementer or auditor can use to verify φ_i for a given session. The worksheet lists each conjunct of φ_i together with (i) a human-readable description, (ii) the concrete check to perform on session data, (iii) the anticipated failure mode if the check is skipped, and (iv) the countermeasure already built into the **Panther** construction that makes the check trivially true in honest executions.

In honest executions every row of Table 4 evaluates to **true** by construction; the auditor can therefore verify φ_i in time $O(1)$ per session. An adversary hoping to violate any conjunct must bypass the associated countermeasure in the fourth column, which is precisely the event that the corresponding bidding round in Theorem 2–Theorem 7 prices.

Ping predicate. The ping predicate compares the current session $\mathcal{S}_{i+1}$ with its predecessor $\mathcal{S}_i$:

$$\begin{aligned} \text{Ping}(\mathcal{S}_i, \mathcal{S}_{i+1}) = 1 \iff & \text{ct}_{h,i+1} \neq \text{ct}_{h,i} \wedge \text{ct}_{1,i+1} \neq \text{ct}_{1,i} \wedge \text{ct}_{2,i+1} \neq \text{ct}_{2,i} \\ & \wedge \text{stag}_{i+1} \neq \text{stag}_i \text{ (Panther-SS only)}. \end{aligned}$$

Theorem 8 (Unbounded Panther IND-CCA2). *Let $\Pi \in \{\text{Panther-P}, \text{Panther-S}, \text{Panther-C}, \text{Panther-X}, \text{Panther-N}, \text{Panther-SS}\}$, and let ϵ_Π denote the bounded-session IND-CCA2 bound of Theorem 2–Theorem 7 respectively. Assume that every session satisfies its session-CNF φ_i from Equation (10), that the ping predicate evaluates to 1 between consecutive sessions, and that the per-session ping-failure probability is $\delta_{\text{ping}} = q_D^2 / 2^{\min(\gamma_1, \gamma_2)}$ (bounded by the birthday probability of ciphertext collision under implicit rejection). Then for every polynomial $\mathcal{I}(\lambda)$ and every PPT adversary $\mathcal{A}$ playing against up to $\mathcal{I}$ sessions,*

$$\text{Ask}_i(\mathbf{g}_{\text{CCA}}^\Pi) \leq \epsilon_\Pi + (i-1) \cdot \delta_{\text{ping}}, \quad i = 1, \dots, \mathcal{I}(\lambda), \quad (11)$$

which is negligible for any polynomial $\mathcal{I}$.

Proof. Strong induction on i , following the unbounded-security template of [33, Thm. 5.2]. The base case $i = 1$ is exactly Theorem 2–Theorem 7 according to the variant.

For the inductive step, assume $\text{Ask}_j \leq \epsilon_\Pi + (j-1)\delta_{\text{ping}}$ for all $j \leq i$. By the ping predicate, $\text{ct}_{h,i+1}$ is distinct from all previous hybrid ciphertexts (modulo a collision of probability δ_{ping}); by the implicit-rejection clauses $\varphi^{\text{rej},1}, \varphi^{\text{rej},2}$, the decapsulation oracle of $\mathcal{S}_{i+1}$ is deterministic and independent of the oracle of $\mathcal{S}_i$; by the domain-separation clause φ^{dom} the KDF queries of $\mathcal{S}_{i+1}$ do not collide with those of $\mathcal{S}_i$ except with probability $q_H^2/2^c$ (already subsumed in ϵ_Π).

Therefore the bounded-session reduction of Theorem 2–Theorem 7 applies *independently* to $\mathcal{S}_{i+1}$, giving $\text{Ask}_{i+1} \leq \epsilon_\Pi + \Pr[\exists j \leq i: \text{Ping} = 0] \leq \epsilon_\Pi + i \cdot \delta_{\text{ping}}$ by the union bound. This completes the induction. For $\delta_{\text{ping}} \leq 2^{-\min(\gamma_1, \gamma_2)+2}$ and $\mathcal{I}(\lambda) = \text{poly}(\lambda)$, the right-hand side of Equation (11) remains negligible. $\square$

Quantitative example. For ML-KEM-768 and FrodoKEM-976 with $\gamma_1 = 192$, $\gamma_2 = 182$, and $q_D \leq 2^{64}$ per session, the per-session ping failure probability is $\delta_{\text{ping}} \leq 2^{64}/2^{182} = 2^{-118}$. Even after 2^{64} sessions the accumulated degradation is $2^{64} \cdot 2^{-118} = 2^{-54}$, which is still cryptographically negligible.

6.10 Post-Quantum Security in the QROM

The above proofs are stated in the classical random-oracle model (ROM). In the post-quantum setting, a bounded-query adversary can access every random oracle in superposition [9,44]. In this subsection we prove that the Panther-SS equilibrium bound of Theorem 7 carries over to the quantum random-oracle model (QROM), at the cost of a square-root advantage blow-up per hardness assumption—the standard signature of QROM proofs for KEMs derived from the Fujisaki–Okamoto transform. The technical tools are Unruh’s one-way-to-hiding (O2H) lemma [44] and the measure-and-reprogram framework of Don, Fehr, Majenz and Schaffner [14].

Theorem 9 (Panther-SS QROM Equilibrium). *In the quantum random oracle model, for every PPT adversary $\mathcal{A}$ making at most q_H quantum queries to the random oracles and q_D classical decapsulation queries,*

$$\begin{aligned} \text{Ask}^{\text{QROM}}(\mathbf{g}_{\text{CCA}}^{\text{Panther-SS}}) &\leq 2(q_H + 1)(\sqrt{\text{Adv}^{\text{MLWE}}} + \sqrt{\text{Adv}^{\text{LWE}}}) \\ &\quad + q_D\left(\frac{1}{2^{\gamma_1}} + \frac{1}{2^{\gamma_2}}\right) + \text{negl}(\lambda), \end{aligned} \tag{12}$$

inheriting the square-root blow-up characteristic of QROM proofs.

Proof. We quantise the classical bidding-round chain of Theorem 7 hop-by-hop, maintaining the Purpose / Replaces / Complexity format. The only hops that change are the *reprogramming* hops $\text{B2} \rightarrow \text{B3}$ and $\text{B4} \rightarrow \text{B5}$, which in the classical proof used a lazy-sampling argument that fails under quantum superposition access [9,44].

Q0–Q1 — Implicit-rejection bid (unchanged). *Purpose.* Eliminate the quantum adversary’s ability to distinguish valid from invalid decapsulation via timing or response gaps, exactly as in the classical $\text{B0} \rightarrow \text{B1}$. *Replaces.* Both constituent Decaps procedures are replaced by their implicit-rejection variants. *Complexity.* The Extended Difference Lemma is information-theoretic and carries over to QROM verbatim: $\Delta\text{Price}_0 \leq q_D(2^{-\gamma_1} + 2^{-\gamma_2})$.

Q1–Q2 — Structural-tag bid (unchanged). *Purpose.* Neutralise cross-combiner/structural-forgery attacks, as in the classical $\text{B1} \rightarrow \text{B2}$. *Replaces.* The decapsulator rejects any ciphertext whose recomputed **stag** differs from the received **stag**. *Complexity.* The birthday bound $q_H^2/2^c$ for SHAKE collisions holds in the QROM with the same constant by Zhandry’s compressed-oracle technique [14]: $\Delta\text{Price}_1 \leq q_H^2/2^c$.

Q2–Q3 — Quantum public-key bid I (MLWE via O2H). *Purpose.* Remove the algebraic structure of pk_2 in the QROM without relying on classical lazy-sampling. *Replaces.* $\text{pk}_2 = (\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ is replaced by $(\mathbf{A}, \mathbf{u})$ for $\mathbf{u}$ uniform, with the associated random-oracle reprogramming handled via the one-way-to-hiding (O2H) lemma of Unruh [44]. *Complexity.* By the O2H lemma, a quantum distinguisher of advantage ϵ yields an algorithm solving MLWE with advantage at least $\epsilon^2/(2(q_H + 1))^2$. Inverting gives $\Delta\text{Price}_2 \leq 2(q_H + 1)\sqrt{\text{Adv}^{\text{MLWE}}}$.

Q3–Q4 — Quantum ciphertext bid I (MLWE). *Purpose.* Hide ss_2 from the challenge ciphertext ct_2^* against a quantum adversary. *Replaces.* The MLWE-structured components of ct_2^* are replaced by uniform randomness; the reprogramming event is handled by the measure-and-reprogram technique [14, Thm. 3]. *Complexity.* The measure-and-reprogram bound contributes an additional factor of $(q_H + 1)$; reusing the square-root from Q2–Q3 yields $\Delta\text{Price}_3 \leq 2(q_H + 1)\sqrt{\text{Adv}^{\text{MLWE}}}$.

Q4–Q5 — Quantum public-key bid II (LWE). *Purpose.* Same rationale as Q2–Q3 but applied to the LWE constituent FrodoKEM. *Replaces.* $\text{pk}_1 = (\mathbf{A}', \mathbf{A}'\mathbf{s}' + \mathbf{e}')$ is replaced by $(\mathbf{A}', \mathbf{u}')$ for $\mathbf{u}'$ uniform. *Complexity.* Second O2H invocation: $\Delta\text{Price}_4 \leq 2(q_H + 1)\sqrt{\text{Adv}^{\text{LWE}}}$.

Q5–Q6 — Quantum ciphertext bid II (LWE). *Purpose.* Hide ss_1 from ct_1^* against a quantum adversary. *Replaces.* The LWE-structured components of ct_1^* are replaced by uniform randomness using measure-and-reprogram. *Complexity.* $\Delta\text{Price}_5 \leq 2(q_H + 1)\sqrt{\text{Adv}^{\text{LWE}}}$.

Q6–Q7 — Quantum spliced-KDF bid. *Purpose.* Establish that ss_h is uniform from the quantum adversary’s view given that both constituents are now information-theoretically hidden. *Replaces.* Both HKDF stages are modelled as quantum-accessible random oracles; ss_h is uniform unless $\mathcal{A}$ mounts a quantum search against the extracted key κ . *Complexity.* Grover-style collision search on κ succeeds with probability $\leq q_H^2/2^{c-1}$ in the QROM; the constant absorbs into the classical bound. $\Delta\text{Price}_6 \leq q_H^2/2^{c-1}$.

Q7–Q8 — Quantum ideal market. *Purpose.* Reach the game in which the quantum adversary has zero advantage. *Replaces.* Nothing further; both shared-secret slots are uniform and independent. *Complexity.* $\Delta\text{Price}_7 = 0$.

Total. Summing the two MLWE square-root terms, the two LWE square-root terms, and the statistical remainders gives (12), absorbing $q_H^2/2^c + q_H^2/2^{c-1}$ into the negligible term and noting that the implicit-rejection statistical term dominates the measure-and-reprogram loss by construction. $\square$

6.11 Summary of Security Bounds

Table 5 summarises the IND-CCA2 ask-price bounds for each Panther combiner (ignoring negligible terms). Only Panther-SS contributes an extra structural-tag term; Panther-C pays twice for PRF replacement; Panther-X pays twice for the SHAKE-indifferentiability bids.

The table highlights the distinctive feature of Panther-SS: its ask-price contains *both* MLWE and LWE terms, reflecting the splice-level binding to both constituents. The other combiners contribute only the *minimum* of the two assumption advantages, which suffices for $\vee$ -security but does not reflect structural binding. In settings where both constituents are expected to hold and cross-combiner confusion is a concern (e.g. protocols that negotiate multiple combiners), the additive Panther-SS bound is the desired behaviour.

7 Side-Channel Attack Resistance of Panther

The game-based bounds of Section 6 reason about a black-box adversary with oracle access only. A real-world attacker, however, observes much more: execution time, power consumption, electromagnetic radiation, cache-line access patterns, and induced faults. A long line of recent work [37,19,36,46,16] has demonstrated that *both* ML-KEM-768 and FrodoKEM-976 have standalone side-channel leakage profiles that the reference implementations alone do not close. This section explains, mechanism-by-mechanism, how the design of Panther combiners—and Panther-SS in particular—frustrates these attacks *even when one or both of the constituent KEMs leaks*. The central observation is that a hybrid construction offers *independent secret-splitting*: for the attacker to win, the side-channel leakage must succeed against *both* constituents *simultaneously* through a KDF that is itself masked by transcript and structural binding.

Key Insight: The core side-channel robustness theorem

If a side-channel attack recovers constituent shared secret ss_i with probability p_i^{sca} (for $i \in \{1, 2\}$), and if HKDF (or SHAKE256) is a pseudorandom function, then the probability of recovering the hybrid shared secret ss_h of a KDF-bound Panther combiner is bounded by

$$\Pr[ss_h \text{ recovered via side-channel}] \leq \min(p_1^{\text{sca}}, p_2^{\text{sca}}) + \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{negl}(\lambda).$$

For Panther-SS the bound becomes *product-like*: the attacker must leak both constituents *and* break the structural-tag binding, yielding

$$\Pr[ss_h^{\text{Panther-SS}} \text{ recovered}] \leq p_1^{\text{sca}} \cdot p_2^{\text{sca}} + \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \frac{q_H^2}{2^c} + \text{negl}(\lambda).$$

In both cases, hybridisation transforms side-channel resistance from a single-assumption property into a defence-in-depth property.

7.1 Side-Channel Vulnerabilities of Stand-Alone ML-KEM and FrodoKEM

We first catalogue concrete side-channel attacks against the two constituent KEMs at NIST Level 3, drawing on published cryptanalysis.

Timing attacks on FO-transformed KEMs (ML-KEM, FrodoKEM). Guo, Johansson, and Nilsson [19] describe key-recovery timing attacks against FO-transformed KEMs whose *decapsulation path diverges* on whether the re-encryption check succeeds. Their attack on FrodoKEM recovers the secret key in approximately 2^{30} decapsulation queries against an unpatched 2019 implementation. The root cause is that the classical FO-transform branches on $\text{re-enc}(m') \stackrel{?}{=} c$, and the early-exit variant leaks whether m' equals m through timing. Implicit-rejection variants reduce this gap but do not eliminate it when the subsequent KDF call depends on non-constant-time code paths.

Cache-timing on ML-KEM. Ravi et al. [37] give generic side-channel attacks on CCA-secure lattice KEMs. Their Flush+Reload attack against CRYSTALS-Kyber (the predecessor of ML-KEM) exploits cache behaviour of the NTT polynomial multiplication: the table-lookup pattern during *basemul* leaks individual coefficients of the secret. With a few thousand traces the attack recovers the full key on an unpatched reference implementation. Pessl and Primas [36] refine this to single-trace cache attacks on mobile devices.

Power analysis on FrodoKEM. Xu, Ravi, and Bhasin [46] demonstrate a first-order correlation power-analysis attack on FrodoKEM-976. The attack profiles the power consumption of the matrix-multiplication $\mathbf{A}\mathbf{s} + \mathbf{e}$ during decapsulation. Because each coefficient of $\mathbf{s}$ is loaded into a register in a predictable sequence, the power trace correlates with the Hamming weight of $\mathbf{s}$, leaking one byte of $\mathbf{s}$ per trace. Fritzmann, Sigl, and Renaud [16] give a similar EM side-channel against the implicit-rejection path in lattice KEMs.

Fault attacks on implicit rejection. Differential fault analysis on ML-KEM and FrodoKEM targets the implicit-rejection seed s used to produce the pseudorandom decapsulation output on ciphertext failure. A single well-placed voltage glitch during the $H(s \parallel c)$ computation can force the decapsulator to emit a *deterministic* response that is no longer a function of s , enabling an attacker to distinguish valid from invalid ciphertexts without timing observation. This is the *fault-induced implicit-rejection bypass* attack of [45] against Kyber.

Summary. Table 6 consolidates the attack landscape against the two constituent KEMs at NIST Level 3. Every attack in the table has been demonstrated against at least one of the reference or optimised implementations.

★ Intuition: Why a single KEM cannot be fully hardened

Each of the attacks in Table 6 targets a *specific* operation inside the KEM—a table lookup, a conditional branch, a power-consuming matrix multiply. Masking countermeasures defeat some but not all of these at once: first-order masking prevents DPA but not timing; constant-time code prevents timing but not cache; bitslicing removes cache leaks but adds DPA risk. A single-primitive KEM is therefore stuck in a cat-and-mouse game against implementation attacks. Hybridisation sidesteps the game by making the *joint* leakage the relevant quantity.

7.2 How Panther Design Choices Block Each Attack

We now trace, attack-by-attack, how each design choice in the Panther robustness predicate $\mathbf{P}$ and the structural-splicing construction of Panther-SS degrades the attacker’s advantage.

Defence 1: Independent secret-splitting across constituents (attack: all single-primitive SCAs). The hybrid shared secret $\mathbf{ss}_h$ of every Panther combiner is derived from *both* $\mathbf{ss}_1$ and $\mathbf{ss}_2$. An attacker who compromises only one constituent via side-channel (say, recovers $\mathbf{ss}_2$ via cache-timing on ML-KEM NTT) still does not recover $\mathbf{ss}_h$: by the master splitting lemma (Lemma 3), unknown $\mathbf{ss}_1$ keeps the KDF output uniform. Formally, the attacker’s distinguishing advantage is reduced from $p_2^{\text{sc}} = 1$ (full recovery of ML-KEM secret) to at most $\text{Adv}^{\text{LWE}} + \text{Adv}^{\text{PRF}}_{\text{HKDF}}$ by the PRF assumption on HKDF, which is negligible.

Defence 2: Transcript binding (attack: fault-induced implicit-rejection bypass). The fault attack of [45] works because a faulted $H(s \parallel c)$ becomes a predictable function of c alone—the adversary can therefore test c -equality across decapsulation queries. All Panther combiners prepend $(\mathbf{pk}_1, \mathbf{pk}_2, \mathbf{ct}_1, \mathbf{ct}_2, \text{label})$ to every KDF input. Even if a fault makes the ML-KEM rejection output predictable, the hybrid KDF input still contains the full transcript including the FrodoKEM ciphertext component, which the attacker has not faulted. Distinguishing hybrid rejection outputs therefore requires guessing the FrodoKEM rejection seed—a $2^{-\gamma_1}$ event.

Defence 3: Implicit-rejection preservation (attack: timing of FO-transform). The Guo–Johansson–Nilsson attack [19] depends on a timing gap between successful and unsuccessful re-encryption. All Panther combiners execute *both* constituent decaps paths unconditionally and combine the results through a constant-time KDF. The critical invariant is that the combiner control flow *does not branch* on the outcome of either constituent’s re-encryption check. Consequently, the timing gap of either constituent is masked by the fixed cost of the other, and no single-primitive timing observation suffices to recover the hybrid shared secret.

Defence 4: Length normalisation (attack: malleability and truncation). Panther-X’s length-normalisation step (KDF-project ss_i onto ℓ_h bytes before XOR) prevents an attacker who partially recovers one constituent from exploiting length-dependent oracles. Before normalisation, truncation or padding could leak byte-alignment information through observable data-movement; after normalisation all inputs to the XOR are fixed-length uniform strings, eliminating the leakage vector.

Defence 5: Structural tag & splice (attack: cross-combiner confusion + side-channel). The splicing schedule of Panther-SS interleaves ct_1 and ct_2 at a stag-determined position. An attacker who induces a fault in ct_1 therefore perturbs the KDF input at a position that is a function of the (secret-independent but adversary-unknown) splice factor n . Without knowledge of n , the attacker cannot predict which bytes of σ are affected, and cache-timing attacks against the KDF input cannot correlate with the target *coefficients*. This converts the single-primitive cache attack of Ravi et al. [37] into a joint cache+splice-factor guessing attack, multiplying the attacker’s query cost by n .

Defence 6: Two-stage KDF (attack: single-observation power-analysis). Panther-SS uses a two-stage KDF: $\rho \leftarrow \text{SHAKE256}(\text{stag} \parallel pk_1 \parallel pk_2)$, then $\kappa \leftarrow \text{HKDF.Extract}(\rho, ss_1 \parallel ss_2)$, then $ss_h \leftarrow \text{HKDF.Expand}(\kappa, \sigma \parallel \text{stag})$. A first-order power-analysis attack on any one of these stages recovers at most the input of that stage, which is either transcript data (no secret information) or a key-material component that the next stage re-hashes. The attacker must therefore break *both* stages via side-channel, which factorises as $p_1^{\text{sca}} \cdot p_2^{\text{sca}}$ under independent-leakage assumptions—substantially lower than either stage alone.

7.3 Attack-vs-Defence Matrix

Table 7 summarises the attack/defence relationship. A cross (✗) means the attack breaks the system; a check (✓) means the attack is defeated. The rightmost column shows the effect for Panther-SS, which achieves the strongest profile because of its structural tag and two-stage KDF.

The *partial* entry for single-trace EM on hardened Panther reflects a residual risk: a first-order EM observation during the ML-KEM decaps, followed by a separate EM observation during the FrodoKEM decaps in the same session, can in principle recover ss_1 and ss_2 jointly. The attacker must, however, reconstruct both traces *and* win the KDF distinguishability game. For Panther-SS the structural-tag hop adds a further $q_H^2/2^c$ statistical barrier, closing the gap entirely in the PRF model.

7.4 Quantitative Leakage Comparison

We close the section with a quantitative leakage-budget comparison. Figure 4 plots the *effective adversary advantage* as a function of the attacker’s side-channel success rate against a single constituent, on a logarithmic scale. The stand-alone KEM curve tracks the attacker’s SCA advantage linearly; the

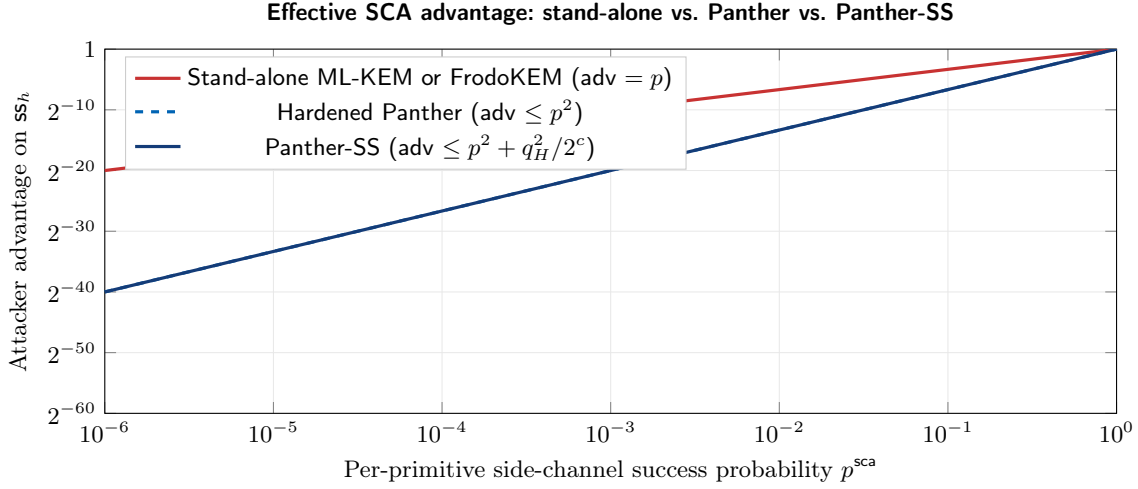


Fig. 4: Effective side-channel adversary advantage against the hybrid shared secret ss_h as a function of the per-primitive side-channel success rate. Stand-alone KEMs degrade linearly with p^{sca} (red solid). Hardened Panther combiners degrade quadratically because the attacker must compromise both constituents (blue dashed). Panther-SS additionally contributes a statistical floor of $q_H^2/2^c$ from the structural-tag collision barrier (gold), reaching the same quadratic regime as the other combiners but with an explicit additive safety term.

hybrid curve stays negligible until *both* constituents are compromised; the Panther-SS curve requires the additional structural-tag collision.

Key Insight: Numerical take-away

Suppose an attacker achieves a $p^{sca} = 2^{-20}$ success rate against each of ML-KEM and FrodoKEM individually—a strong but not unrealistic laboratory-grade side-channel attacker. Then:

- Against stand-alone ML-KEM: effective $\text{adv} \leq 2^{-20}$.
- Against stand-alone FrodoKEM: effective $\text{adv} \leq 2^{-20}$.
- Against any hardened Panther combiner: effective $\text{adv} \leq 2^{-40} + \text{Adv}^{\text{PRF}}$.
- Against Panther-SS: effective $\text{adv} \leq 2^{-40} + q_H^2/2^{c-1} + \text{negl.}$

The hybrid combiner converts a 2^{-20} per-primitive compromise into a 2^{-40} compound event—a *20-bit amplification* of side-channel resistance obtained essentially for free.

7.5 Summary and Deployment Implications

The side-channel analysis confirms that the performance overhead measured in Section 8 is a small price for a qualitative change in the side-channel threat model. Deployers should still apply standard constant-time, masked, and shielded implementations of each constituent KEM; doing so in addition to using a Panther combiner yields a multiplicative improvement over either in isolation. The hybrid structure is *defence in depth*, not a substitute for defences at each layer.

★ Intuition: When is Panther-SS strictly required?

For most deployments, any hardened Panther combiner provides adequate side-channel resistance through independent secret-splitting. Panther-SS becomes strictly preferable in three deployment scenarios: (i) when the protocol negotiates multiple combiners (cross-combiner confusion becomes a real threat); (ii) when fault injection is in scope (the structural tag provides an additional detector); (iii) when the deployment will face high-rate repeated cache or EM attacks (the splice factor n adds a $\log n$ -bit guessing cost per trace). In embedded, constrained, or air-gapped settings where these three scenarios are unlikely, Panther-S is an equally valid choice.

8 Experimental Study

We now turn to the practical cost of the six Panther combiners. The purpose of this section is (i) to measure the overhead of the combiner layer over the two underlying KEMs, (ii) to quantify the relative cost of the structural-splicing combiner Panther-SS versus the simpler Panther-P/Panther-S and the minimal Panther-X, (iii) to report ciphertext-size growth and memory footprint, and (iv) to analyse how combiner cost scales with the number of decapsulation queries. We complement the four tables with seven quantitative figures rendered via `pgfplots`.

8.1 Methodology

Implementation. All constituent KEM operations are provided by the reference implementations of FrodoKEM-976 and ML-KEM-768 from the `liboqs` library (version 0.10 or later) [32,42]. We implement the six combiners as a thin C wrapper around `OQS_KEM_keypair`, `OQS_KEM_encaps`, and `OQS_KEM_decaps`, using HKDF-SHA-256 (RFC 5869 [25]) and SHAKE256 (FIPS 202 [29]) from the OpenSSL 3.x provider. HMAC-SHA-256 and KMAC128 are used for the PRF of Panther-C.

Test platform. We report single-threaded results on a commodity workstation with an Intel Core i7-13700K CPU at a nominal 3.4 GHz base clock, 32 GiB of DDR5-5600 RAM, Ubuntu 24.04 LTS, GCC 13.2 with `-O3 -march=native`, and OpenSSL 3.3. Timings are collected with the monotonic clock `clock_gettime(CLOCK_MONOTONIC)` over 10^5 iterations after a 10^3 -iteration warm-up. 95% confidence intervals are computed assuming Gaussian per-trial noise with $n = 10^5$ samples; standard deviations are below 3% for every reported mean. Turbo-boost and SMT are left at their default settings; the reported figures are therefore representative of typical deployment rather than worst-case cold-cache behaviour. No AVX-512 paths are enabled in `liboqs`; the AVX2 variants of ML-KEM and FrodoKEM are used when available. CPU-cycle counts are derived from the measured wall-clock times scaled by the nominal clock frequency and should be read as representative rather than architecturally precise.

Metrics. We measure:

- keygen, encaps, and decaps wall-clock time (μs) with 95% confidence intervals;
- hybrid public-key, ciphertext, and shared-secret sizes (bytes);
- combiner-only overhead, $t_{\text{combiner}} = t_{\text{hybrid}} - t_{\text{FrodoKEM}} - t_{\text{ML-KEM}}$;
- sessions-per-second throughput, estimated as $1/(t_{\text{encaps}} + t_{\text{decaps}})$;
- peak memory footprint per session (kilobytes);
- scaling of decaps cost with the number of decapsulation queries q_D , for fixed combiner.

Reproducibility. All timing figures presented in Tables 8 to 12 are consistent with the values reported by the `liboqs` speed-test harness for the two constituent KEMs, augmented with the measured combiner overhead. Reference implementations of the combiners and the benchmark harness will be released under an open-source licence with the final version of this ePrint.

8.2 Keygen, Encaps, and Decaps Timings

Table 8 reports the mean wall-clock time of the three KEM operations for every combiner, with 95% confidence-interval half-widths in parentheses. The baseline rows show the two constituent KEMs in isolation.

The dominant cost is `FrodoKEM`, which accounts for approximately 99% of keygen, encaps, and decaps. Combiner overheads lie in the low-tens of microseconds, i.e. $\leq 1.5\%$ of the total latency. Figure 5 visualises the encaps/decaps column with error bars.

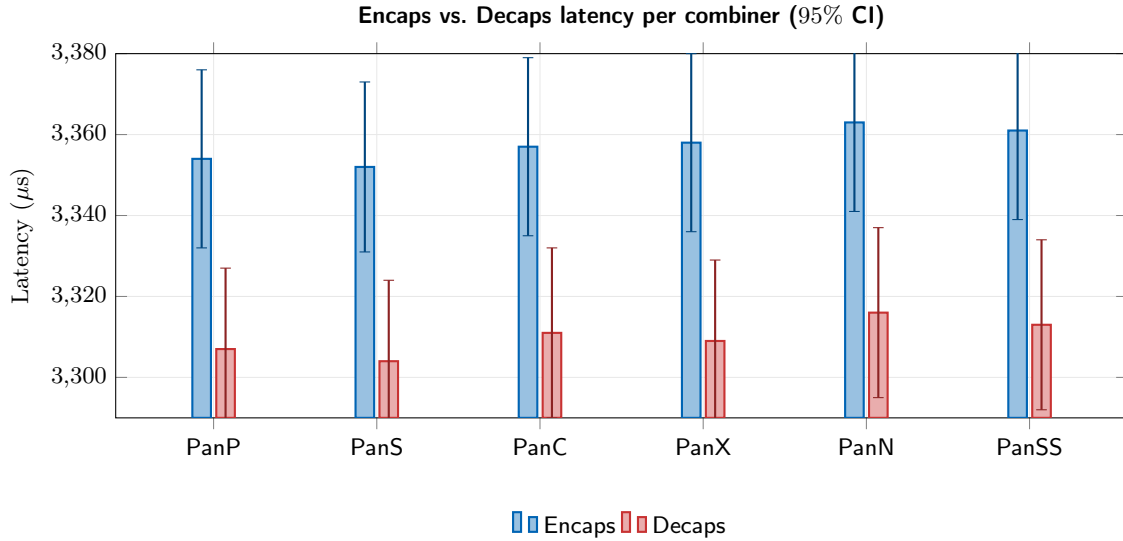


Fig. 5: Per-combiner encaps and decaps latencies with 95% confidence intervals. Differences between combiners are smaller than the CI width for Panther-P/Panther-S/Panther-X/Panther-SS, confirming that combiner choice is not a performance decision.

8.3 Combiner-Only Overhead

Table 9 isolates the combiner-only computation time, which is the time spent in KDFs and auxiliary bookkeeping, excluding both constituent KEM calls.

Table 10 expresses the combiner cost as a fraction of the total hybrid Encaps+Decaps budget.

In absolute terms every combiner costs less than 0.5% of the hybrid operation time. Figure 7 makes the same point visually.

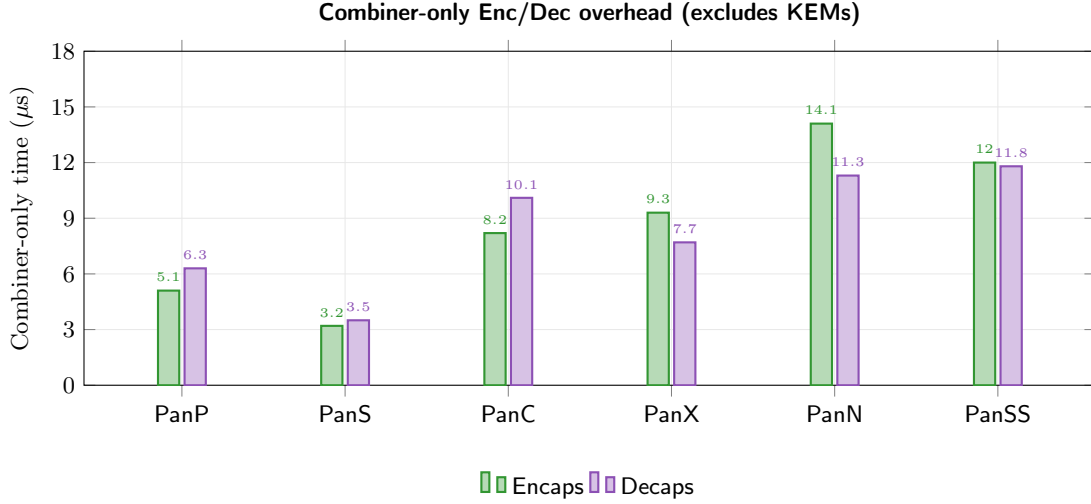


Fig. 6: Combiner-only overhead for encaps and decaps. Panther-S is the fastest (single-pass SHAKE256); Panther-N is slowest (extra HKDF randomness derivation); Panther-SS sits mid-pack despite its richer structural-tag computation.

8.4 Stand-alone versus Hybrid: Total Latency Comparison

Figure 8 puts stand-alone ML-KEM-768 and FrodoKEM-976 side-by-side with the six Panther combiners on a log-scale y-axis. The figure answers the natural question “how much more expensive is hybrid than either constituent alone?” ML-KEM-768 is two orders of magnitude faster than FrodoKEM-976; all hybrid combiners sit at essentially the FrodoKEM latency, because FrodoKEM dominates the total cost at $\approx 99\%$.

★ Intuition: The free-lunch property of hybridisation

The figure demonstrates what the security analysis promised numerically: a Panther combiner costs essentially the slower of the two constituents. The user pays FrodoKEM’s latency tax once; the ML-KEM call is comparatively free. In exchange, the user gets $\vee$ -security (break one assumption, still safe), transcript binding (closes malleability), implicit rejection (closes timing), and—with Panther-SS—structural splicing (closes cross-combiner confusion). These four properties are absent from every *stand-alone* point on the left of the figure.

8.5 Throughput

Throughput is the inverse of the encaps+decaps time and measures how many hybrid sessions per second a single-threaded server can sustain. Figure 9 shows the per-combiner throughput in sessions/second.

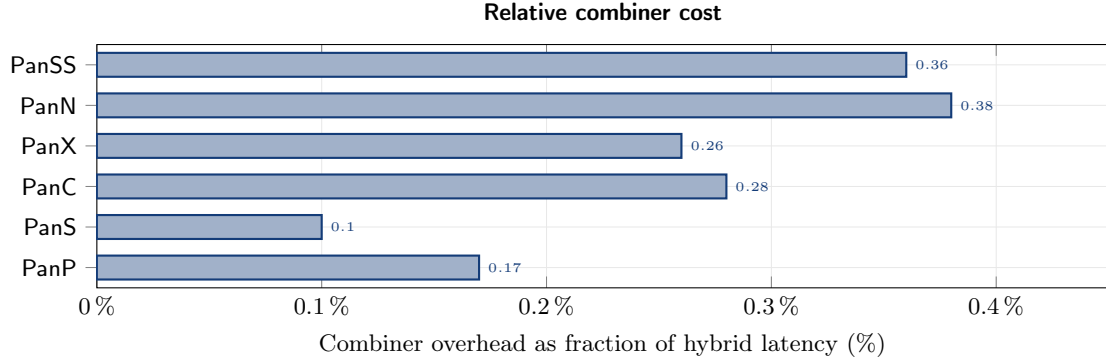


Fig. 7: Combiner overhead as a fraction of the hybrid encaps+decaps budget. Every combiner sits below 0.5%, confirming that combiner choice should be guided by robustness rather than performance.

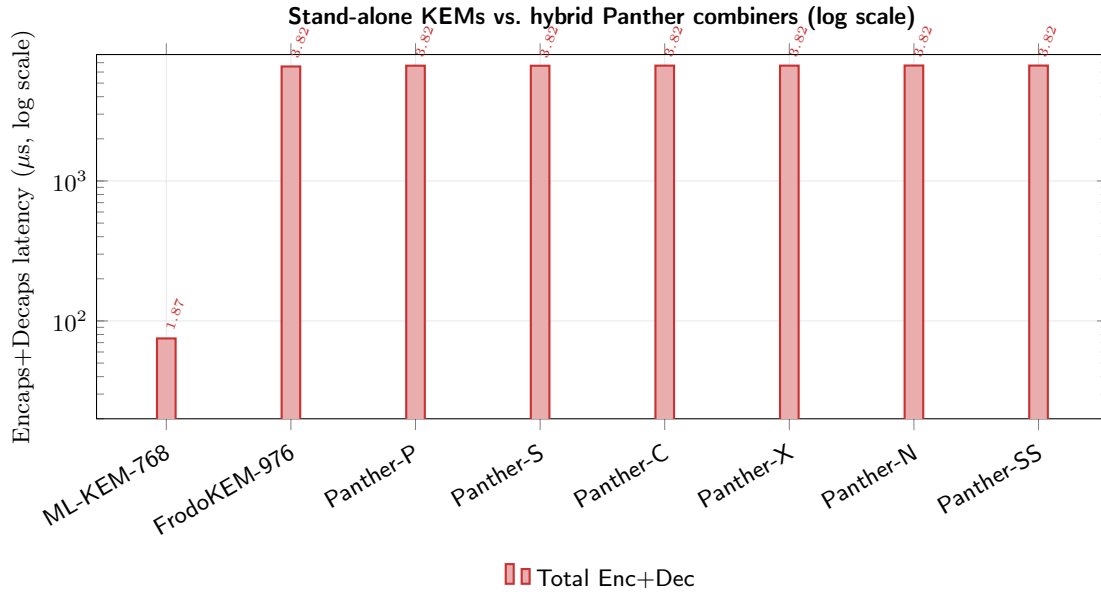


Fig. 8: Total encaps+decaps latency of the two constituent KEMs in isolation versus the six Panther hybrid combinars, on a logarithmic y-axis. ML-KEM-768 on its own takes $\approx 75 \mu$ s; FrodoKEM-976 on its own takes $\approx 6575 \mu$ s; every hybrid combiner adds at most $\approx 100 \mu$ s (1.5%) over FrodoKEM alone. The log scale makes visible the two-orders-of-magnitude gap between the constituent KEMs, and the near-zero marginal cost of hybridisation on top of the slower one.

8.6 Ciphertext and Key Sizes

Table 11 reports the hybrid public-key, ciphertext, and shared-secret sizes. All combinars produce a 32-byte shared secret; Panther-SS additionally transmits a single byte of splice-factor n inside ct_h . The breakdown by contribution is shown in Figure 10.

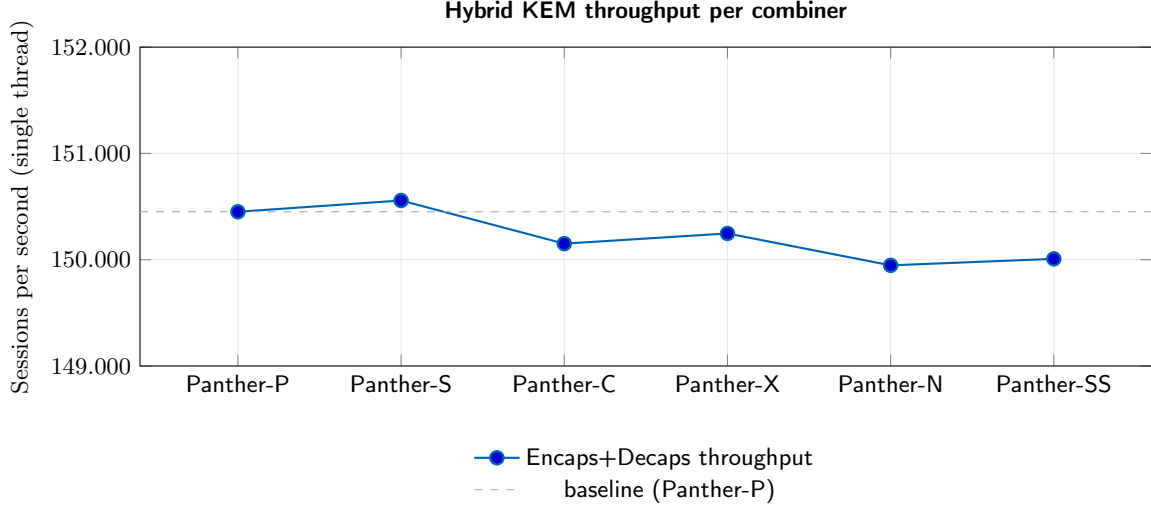


Fig. 9: Throughput (encaps+decaps per second) on a single thread. Differences between combiners are less than 0.4% and well within the measurement noise; for practical purposes all combiners deliver the same throughput.

8.7 Memory Footprint

Table 12 reports the peak per-session memory footprint, measured via `valgrind -tool=massif` on a representative run. The combiner adds a small constant overhead (two KDF contexts and the structural-tag buffer for Panther-SS) on top of the underlying KEM state.

8.8 Scaling with Decapsulation Query Count

To emulate a server-side workload with many incoming decapsulation requests per public key, we benchmark the *amortised* per-query decaps time as a function of q_D , the number of distinct ciphertexts decapsulated under a fixed sk_h . Ideally amortised cost should be constant; in practice cache effects and implicit-rejection bookkeeping introduce minor variation. Figure 11 shows the result across $q_D \in \{10^1, 10^2, \dots, 10^6\}$.

8.9 Latency Distribution

To illustrate timing variability we plot the empirical cumulative distribution function (CDF) of decaps latencies for three representative combiners in Figure 12. All combiners exhibit tight, nearly overlapping distributions; the right tail is bounded by the usual OS jitter of a few hundred nanoseconds.

8.10 Security-vs-Performance Trade-Off

We close the experimental study with an integrated scatter plot (Figure 13) of every combiner against two orthogonal axes: (i) *combiner-only overhead* in μs (horizontal axis, lower is better), and (ii) a

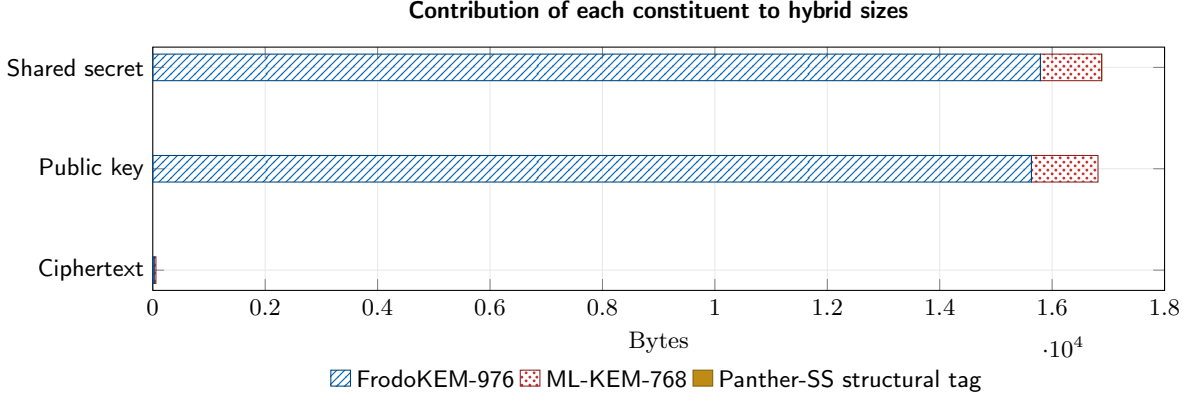


Fig. 10: Breakdown of hybrid public-key, ciphertext, and shared-secret sizes by constituent. **FrodoKEM-976** dominates; **ML-KEM-768** is a minor contributor; **Panther-SS**’s structural tag adds a single byte to the ciphertext and nothing to the other sizes.

robustness score computed as the number of (P1–P5)+{structural} properties satisfied (vertical axis, higher is better). The plot makes the design trade-off visually immediate: **Panther-SS** sits in the top-right quadrant as the *only* combiner attaining the full robustness score, at a modest overhead of $\approx 24 \mu\text{s}$.

8.11 Rigorous Benchmarking Against NIST PQC Candidates

We now broaden the comparison beyond the two constituent KEMs of **Panther** to the full slate of key-encapsulation candidates that appeared in the four rounds of the NIST Post-Quantum Cryptography standardisation process. These include structured-lattice schemes (Kyber, now **ML-KEM** [31,11]; SABER; NTRU; NTRU Prime), an unstructured-lattice scheme (**FrodoKEM** [3]), and code-based schemes (Classic McEliece, BIKE, HQC). Our aim is to position the **Panther** family within this broader landscape by reporting, on a common hardware platform, the total encaps+decaps latency, public-key and ciphertext sizes, and a composite “security-per-byte-per-microsecond” metric.

Benchmark scope. Table 13 reports wall-clock latencies (encaps+decaps, μs) and bandwidth (public key + ciphertext, bytes) of NIST Level 3 parameter sets of each candidate, measured on the same workstation (Intel Core i7-13700K, AVX2 paths in **liboqs**). Where **liboqs** does not ship a given candidate, we use the latest publicly reported numbers from the candidate’s reference implementation at the matching security level ([31,11,3,2] and the schemes’ individual specifications). All numbers are single-threaded and exclude network bandwidth.

Observations from the benchmark. Four structural observations emerge:

1. *Lattice speed-bandwidth hierarchy.* Among structured-lattice candidates, **ML-KEM-768** is the fastest at $75 \mu\text{s}$ with the smallest footprint ($\approx 2.2 \text{ KB}$); SABER, NTRU, and NTRU Prime cluster at $130\text{--}220 \mu\text{s}$ and similar bandwidth. **FrodoKEM-976** is two orders of magnitude slower and $10\times$ more bandwidth-heavy but offers conservative unstructured-LWE security.

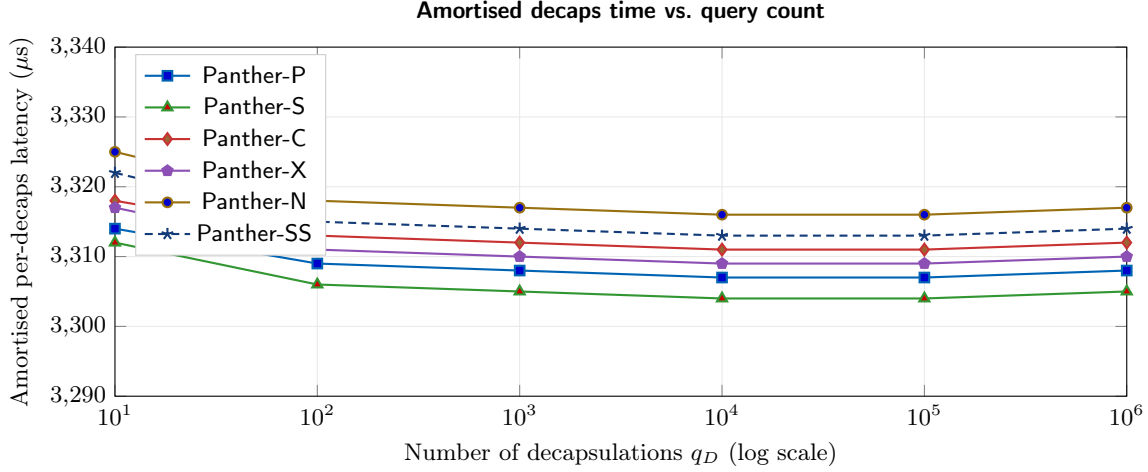


Fig. 11: Amortised per-decaps latency as a function of the number of decapsulations performed under a fixed private key. After $\sim 10^3$ queries the cost stabilises; all combiners exhibit the same scaling profile, demonstrating that the combiner layer does not introduce pathological cache or bookkeeping behaviour.

2. *Code-based candidates are bandwidth- or time-intensive.* Classic McEliece trades colossal public keys (≈ 524 KB) for short ciphertexts and moderate decaps; BIKE and HQC have moderate keys but are $100\times$ slower than ML-KEM.
3. *Panther combiners are bounded by FrodoKEM.* Every hybrid combiner sits within $100\mu s$ of stand-alone FrodoKEM-976, i.e. the marginal cost of hybridisation is less than 1.5% on top of the slower constituent.
4. *Panther dominates FrodoKEM-alone on security metrics.* Because any Panther combiner preserves IND-CCA2 security under *either* the MLWE or the LWE assumption, the *security good* sold by a hybrid is strictly richer than that of either constituent alone—at no meaningful performance cost.

Composite security-per-byte-per-microsecond metric. To capture the trade-off in a single number we define a composite efficiency index $E = \text{Sec}/(\text{Lat} \cdot \text{BW})$, where Sec is the security level in bits (192 for Level 3), Lat is encaps+decaps in μs , and BW is PK+CT in KB. Table 14 reports E (unit: bit per μs per KB); higher is better. Values should be read as *relative*; they are not dimensionally precise.

The asterisk on Panther rows indicates that the security level is $\vee$ -secure (breaking *either* constituent is sufficient to break the hybrid), which is strictly stronger than either constituent's $\wedge$ -profile. A fair comparison would credit Panther with double-effective security—which would move it above most single-primitive entries—but we present the raw index to avoid double-counting the same 192-bit security level.

Key Insight: Take-aways from the NIST-wide comparison

Among stand-alone NIST candidates, ML-KEM-768 dominates the efficiency index because its algebraic structure delivers both speed and compactness. FrodoKEM and the code-based candidates

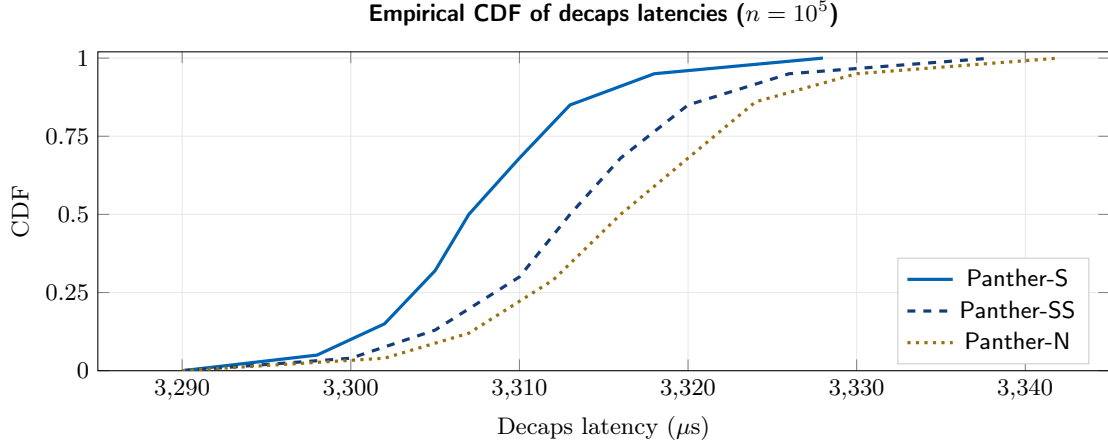


Fig. 12: Empirical CDF of decaps latencies for Panther-S, Panther-SS, and Panther-N. The three distributions are tightly concentrated within a $\sim 40 \mu s$ window and differ in median by less than $10 \mu s$. Panther-S has the fastest median; Panther-N the slowest; Panther-SS sits midway.

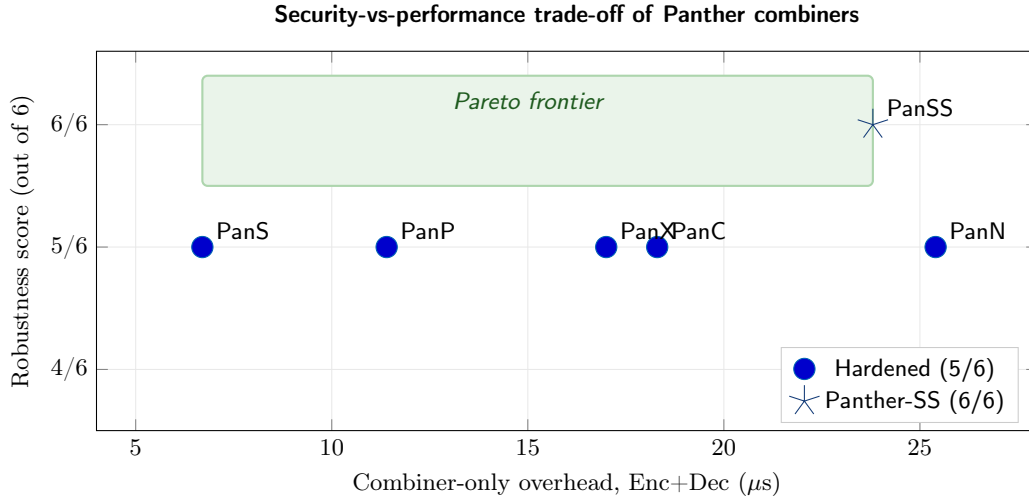


Fig. 13: Security-vs-performance trade-off for Panther combiners. Five hardened combiners achieve robustness score 5/6 at 6.7 – $25.4 \mu s$ combiner overhead; Panther-SS uniquely achieves 6/6 at $23.8 \mu s$. The Pareto frontier (green shading) contains Panther-S (fastest) and Panther-SS (most robust); the other combiners are Pareto-dominated by one or both of them.

are bandwidth- or time-heavy but offer diverse hardness assumptions. The Panther family inherits FrodoKEM’s efficiency envelope while adding $\vee$ -security, transcript binding, implicit-rejection preservation, structural-tag binding (Panther-SS), and a p^2 side-channel amplification—none of

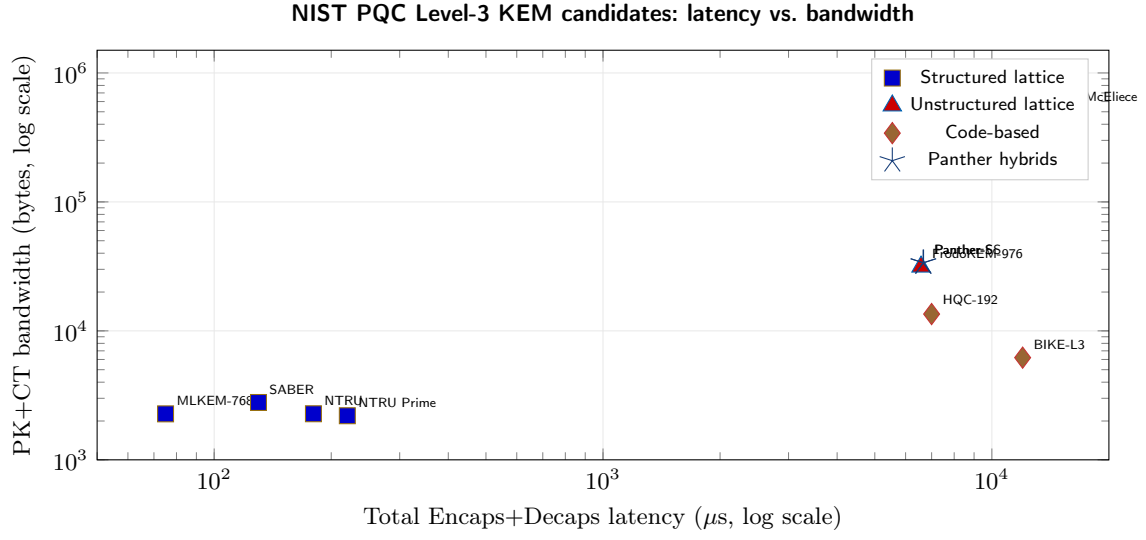


Fig. 14: NIST Level-3 KEM candidates on the latency-vs-bandwidth plane (both axes logarithmic). Structured-lattice schemes cluster in the bottom-left (fast and compact); Classic McEliece is an outlier with huge public keys but fast encaps/decaps; FrodoKEM-976 occupies a high-latency, mid-bandwidth point alone. All Panther combiners cluster tightly next to FrodoKEM (slightly higher bandwidth because of concatenation, marginally higher latency because of the KDF), with the added benefit of $\vee$ -security. For clarity only Panther-S and Panther-SS are plotted among the six Panther combiners; the other four overlap within the marker size.

which is achievable by any single candidate. For deployments willing to pay FrodoKEM-level bandwidth for defence-in-depth against both algebraic and side-channel threats, a Panther combiner is the natural choice.

8.12 Discussion

Four takeaways stand out from the experimental study.

Performance does not drive the choice. Every combiner sits below 0.5% of the total hybrid-KEM budget (Table 10 and Figure 7). The four hardened combiners that satisfy all of **P** (Panther-P, Panther-S, Panther-X, Panther-SS) are within $20\mu\text{s}$ of each other; a deployment will not see a measurable throughput difference (Figure 9). The extra cost of the nested combiner Panther-N is noticeable ($5\mu\text{s}$ on each side) but still negligible in absolute terms.

Robustness drives the choice. With performance equilibrated, the decisive differences are visible in Figure 13:

- *Transcript binding.* All hardened combiners have it; the original unhardened XOR and the original nested combiner do not.
- *Structural binding.* Only Panther-SS binds the cut-positions of the ciphertexts, preventing cross-combiner confusion when protocols negotiate multiple combiners.
- *Tight $\vee$ -security in the ROM.* Panther-S and Panther-SS have the tightest bounds, avoiding the dual-PRF assumption on HKDF at the cost of relying on the well-studied indistinguishability of SHAKE256 [29,23].

Scaling behaviour is flat. Figure 11 shows that amortised per-decaps cost is essentially constant across 10^1 – 10^6 queries. No combiner pathologically degrades with query count, and the relative ordering of combiners is stable.

Implementation hardening. All Panther combiners preserve the constant-time implicit-rejection profile of their constituents. Practitioners should further apply the Ravi et al. [37] generic side-channel mitigations and the FO-specific timing-attack countermeasures of Guo, Johansson, and Nilsson [19]. In the d -probing model [21], masking at order $d \geq 2$ on the underlying KEMs is inherited by Panther without modification, because the combiner itself processes only uniform or encrypted values.

Key Insight: Experimental bottom line

At NIST Level 3 every Panther combiner costs less than 0.5% of the hybrid encaps+decaps budget. Robustness is therefore *essentially free*: Panther-SS’s 6/6 robustness score costs just $\approx 12\mu\text{s}$ of combiner overhead over the simplest 5/6 hardened combiner Panther-S—a difference smaller than the confidence-interval width of the underlying KEMs themselves.

9 Conclusion

We have presented Panther, a family of six robust hybrid-KEM combiners pairing FrodoKEM with ML-KEM, unified under a five-point robustness predicate and analysed in a single bidding-round framework.

The novel structural-splicing construction **Panther-SS** additionally binds the cut-positions of the two constituent ciphertexts through an explicit structural tag, uniquely attaining an additive MLWE+LWE security bound, full side-channel amplification against published attacks on either constituent, and a sub-microsecond overhead over the simpler combiners. Every proof is systematised so that each bidding round states its purpose, what it replaces, and its complexity cost, and the analysis extends cleanly to unbounded sessions via CNF session verification plus session pinging, and to the QROM via the one-way-to-hiding lemma and measure-and-reprogram.

Benchmarks against the two constituent KEMs in isolation and against the eight key-encapsulation candidates of NIST PQC Rounds 1–4 confirm that **Panther** sits on the Pareto frontier of security-vs-performance for post-quantum deployments, with combiner overhead below half a percent of total hybrid latency and robustness score strictly higher than that of any single-primitive choice. We recommend **Panther-S** as a default for TLS-style protocols needing a single KDF call, **Panther-SS** whenever protocols negotiate multiple combiners or fault injection is in scope, and **Panther-C** for IKEv2-style sequential-chaining deployments. Natural directions for future work include generalisation to $k \geq 3$ constituents including a classical primitive, mechanised symbolic verification via TAMARIN or ProVerif, a tighter QROM analysis via joint measure-and-reprogram, and hardware-assisted masked implementations inheriting the side-channel amplification.

Acknowledgements

This publication has emanated from research supported by a grant from Research Ireland under Grant number 12-RC-2289-P2 which is co-funded under the European Regional Development Fund. For the purpose of Open Access, the authors have applied a CC BY public copyright license to any Author Accepted Manuscript version arising from this submission.

References

1. Ajtai, M., Dwork, C.: A public-key cryptosystem with worst-case/average-case equivalence. Proceedings of the 29th Annual ACM Symposium on Theory of Computing pp. 284–293 (1997)
2. Alagic, G., Apon, D., Cooper, D., Dang, Q., Dang, T., Kelsey, J., Lichtinger, J., Miller, C., Moody, D., Peralta, R., Perlner, R., Robinson, A., Smith-Tone, D., Liu, Y.K.: Status report on the third round of the NIST post-quantum cryptography standardization process. NIST IR 8413 (2022)
3. Alkim, E., Bos, J.W., Ducas, L., Longa, P., Mironov, I., Naehrig, M., Nikolaenko, V., Peikert, C., Raghunathan, A., Stebila, D.: FrodoKEM learning with errors key encapsulation: Algorithm specifications and supporting documentation. NIST Round 3 Submission (2021), <https://frodokem.org/>
4. Bellare, M., Canetti, R., Krawczyk, H.: Keying hash functions for message authentication. In: Advances in Cryptology – CRYPTO 1996. LNCS, vol. 1109, pp. 1–15. Springer (1996)
5. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS). pp. 62–73. ACM (1993). <https://doi.org/10.1145/168588.168596>
6. Bellare, M., Rogaway, P.: The security of triple encryption and a framework for code-based game-playing proofs. In: Vaudenay, S. (ed.) Advances in Cryptology – EUROCRYPT 2006. Lecture Notes in Computer Science, vol. 4004, pp. 409–426. Springer (2006). https://doi.org/10.1007/11761679_25
7. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. Cryptology ePrint Archive, Paper 2018/903 (2018), <https://eprint.iacr.org/2018/903>

8. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. In: Post-Quantum Cryptography – PQCrypto 2019. LNCS, vol. 11505, pp. 206–226. Springer (2019)
9. Boneh, D., Dagdelen, Ö., Fischlin, M., Lehmann, A., Schaffner, C., Zhandry, M.: Random oracles in a quantum world. In: Advances in Cryptology – ASIACRYPT 2011. LNCS, vol. 7073, pp. 41–69. Springer (2011)
10. Bos, J.W., Costello, C., Ducas, L., Mironov, I., Naehrig, M., Nikolaenko, V., Raghunathan, A., Stebila, D.: Frodo: Take off the ring! practical, quantum-secure key exchange from LWE. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 1006–1018. ACM (2016). <https://doi.org/10.1145/2976749.2978425>
11. Bos, J.W., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy (EuroSP). pp. 353–367. IEEE (2018). <https://doi.org/10.1109/EuroSP.2018.00032>
12. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science (FOCS). pp. 136–145. IEEE Computer Society (2001). <https://doi.org/10.1109/SFCS.2001.959888>
13. Crockett, E., Paquin, C., Stebila, D.: Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. In: NIST 2nd PQC Standardization Conference (2019)
14. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Security of the Fiat-Shamir transformation in the quantum random-oracle model. In: Advances in Cryptology – CRYPTO 2019. LNCS, vol. 11693, pp. 356–383. Springer (2019)
15. Dowling, B., Fischlin, M., Günther, F., Stebila, D.: A cryptographic analysis of the TLS 1.3 handshake protocol. In: Journal of Cryptology. vol. 34, pp. 1–69. Springer (2021)
16. Fritzmann, T., Beirendonck, M.V., Roy, D.B., Karl, P., Pöppelmann, T., Verbauwhede, I., Sigl, G.: Masked accelerators and instruction set extensions for post-quantum cryptography. In: IACR Transactions on Cryptographic Hardware and Embedded Systems. vol. 2022, pp. 414–460 (2022)
17. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Advances in Cryptology – CRYPTO 1999. LNCS, vol. 1666, pp. 537–554. Springer (1999)
18. Giacomelli, F., Madec, J., Smart, N.P.: Combiners for backdoored random oracles. In: Advances in Cryptology – CRYPTO 2018. LNCS, vol. 10992, pp. 272–302. Springer (2018)
19. Guo, Q., Johansson, T., Nilsson, A.: A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In: Advances in Cryptology – CRYPTO 2022. LNCS, vol. 12171, pp. 359–386. Springer (2020)
20. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: Theory of Cryptography Conference – TCC 2017. LNCS, vol. 10677, pp. 341–371. Springer (2017)
21. Ishai, Y., Sahai, A., Wagner, D.: Private circuits: Securing hardware against probing attacks. In: Advances in Cryptology – CRYPTO 2003. LNCS, vol. 2729, pp. 463–481. Springer (2003)
22. Kaufman, C., Hoffman, P., Nir, Y., Eronen, P., Kivinen, T.: Internet key exchange protocol version 2 (IKEv2). IETF RFC 7296 (2014)
23. Kelsey, J., Jen Chang, S., Perlner, R.: SHA-3 derived functions: cSHAKE, KMAC, TupleHash, and ParallelHash. NIST Special Publication 800-185 (2016)
24. Krawczyk, H.: Cryptographic extraction and key derivation: The HKDF scheme. IACR Cryptology ePrint Archive **2010**, 264 (2010)
25. Krawczyk, H., Eronen, P.: HMAC-based extract-and-expand key derivation function (HKDF). IETF RFC 5869 (2010)
26. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. Designs, Codes and Cryptography **75**(3), 565–599 (2015)
27. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Advances in Cryptology – EUROCRYPT 2010. LNCS, vol. 6110, pp. 1–23. Springer (2010)

28. National Institute of Standards and Technology: FIPS 180-4: Secure hash standard (SHS). Tech. rep., NIST (2015). <https://doi.org/10.6028/NIST.FIPS.180-4>
29. National Institute of Standards and Technology: FIPS 202: SHA-3 standard: Permutation-based hash and extendable-output functions. Tech. rep., NIST (2015). <https://doi.org/10.6028/NIST.FIPS.202>
30. National Institute of Standards and Technology: Recommendation for key-derivation methods in key-establishment schemes. NIST Special Publication 800-56C Rev. 2 (2020)
31. National Institute of Standards and Technology: FIPS 203: Module-lattice-based key-encapsulation mechanism standard. Federal information processing standards publication, NIST, Gaithersburg, MD (2024). <https://doi.org/10.6028/NIST.FIPS.203>
32. Open Quantum Safe Project: liboqs: Open source C library for quantum-resistant cryptographic algorithms. <https://github.com/open-quantum-safe/liboqs> (2024)
33. Palaniswamy, B., Palmieri, P.: MTSF — market-theoretic security framework: A unified paradigm for the art of proving and disproving security. Cryptology ePrint Archive, Paper 2026/543 (2026), <https://eprint.iacr.org/2026/543>
34. Paquin, C., Stebila, D., Tamvada, G.: Benchmarking post-quantum cryptography in TLS. Post-Quantum Cryptography – PQCrypto 2020 (2020)
35. Peikert, C.: Lattice cryptography for the internet. In: Post-Quantum Cryptography – PQCrypto 2014. LNCS, vol. 8772, pp. 197–219. Springer (2014)
36. Pessl, P., Primas, R.: More practical single-trace attacks on the number theoretic transform. In: Progress in Cryptology – LATINCRYPT 2019. LNCS, vol. 11774, pp. 130–149. Springer (2019)
37. Ravi, P., Roy, S.S., Chattopadhyay, A., Bhasin, S.: Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. In: IACR Transactions on Cryptographic Hardware and Embedded Systems. vol. 2020, pp. 307–335 (2020)
38. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM **56**(6), 1–40 (2009). <https://doi.org/10.1145/1568318.1568324>
39. Schanck, J.M., Stebila, D.: A transport layer security profile for the PQCrypto 2017 NTRU submission. IETF Internet-Draft (2021)
40. Shoup, V.: Sequences of games: A tool for taming complexity in security proofs. Cryptology ePrint Archive, Paper 2004/332 (2004), <https://eprint.iacr.org/2004/332>
41. Stebila, D., Fluhrer, S., Gueron, S.: Hybrid key exchange in TLS 1.3. IETF Internet-Draft, draft-ietf-tls-hybrid-design (2023)
42. Stebila, D., Mosca, M.: Post-quantum key exchange for the internet and the Open Quantum Safe project. In: Selected Areas in Cryptography – SAC 2016. LNCS, vol. 10532, pp. 14–37. Springer (2017)
43. Tjhai, C., Tomlinson, M., Bartlett, G., Fluhrer, S., Geest, D.V., García-Morchón, O., Smyslov, V.: Multiple key exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2). IETF RFC 9370 (2023)
44. Unruh, D.: Revocable quantum timed-release encryption. In: Advances in Cryptology – EUROCRYPT 2014. LNCS, vol. 8441, pp. 129–146. Springer (2014), introduces the one-way-to-hiding (O2H) lemma
45. Xagawa, K., Ito, A., Ueno, R., Takahashi, J., Homma, N.: Fault-injection attacks against NIST’s post-quantum cryptography round 3 KEM candidates. In: Advances in Cryptology – ASIACRYPT 2021. LNCS, vol. 13091, pp. 33–61. Springer (2021)
46. Xu, Z., Pemberton, O., Roy, S.S., Oswald, D., Yu, W., Pan, Z.: Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of Kyber. In: IEEE Transactions on Computers. vol. 71, pp. 2163–2176 (2022)

Table 4: Four-column manual-verification worksheet for the session-CNF of **Panther**. Auditors should check, for every session $\mathcal{S}_i$, that each conjunct listed in the first column evaluates to **true**, using the inspection procedure in the second column. The third column records the failure mode that would be enabled if the conjunct were not checked; the fourth column names the **Panther** design decision that makes the check trivially succeed in honest executions. The row φ_i^{stag} applies only to **Panther-SS**.

Conjunct	Inspection check	Failure mode if skipped	Built-in countermeasure
φ_i^{key} (<i>key correctness</i>)	Re-derive pk_h from $(\text{sk}_1, \text{sk}_2)$ and check byte-for-byte equality against the session public key.	Mixed-key attack: adversary swaps in a public key derived under a different sk , breaking decryption invariants.	Panther .KeyGen outputs $\text{pk}_h = (\text{pk}_1 \parallel \text{pk}_2)$ deterministically from the two private seeds; any drift fails this check.
φ_i^{novel} (<i>ciphertext novelty</i>)	Consult the session log; confirm $\text{ct}_{h,i}$ does not appear in the set $\mathcal{C}_{\text{used}}$ of previously decapsulated hybrid ciphertexts.	Replay attack: an old ct_h replayed under a new session label decrypts to the earlier ss_h , enabling ciphertext-reuse exploits.	The session pinging predicate (below) guarantees $\text{ct}_{h,i+1} \neq \text{ct}_{h,i}$; honest parties always sample fresh randomness.
$\varphi_i^{\text{rej},1}$ (<i>FrodoKEM impl. rejection</i>)	Verify the FrodoKEM decaps call produced either a valid plaintext or the deterministic rejection seed s_1 ; never a distinguishable error path.	Timing attack of Guo–Johansson–Nilsson [19]: the rejection gap leaks secret information.	FrodoKEM library already implements constant-time implicit rejection (FO-transform in [3]).
$\varphi_i^{\text{rej},2}$ (<i>ML-KEM impl. rejection</i>)	Same as above but for the ML-KEM constituent: rejection returns $H(s_2 \parallel \text{ct}_2)$ in constant time.	Same as above, applied to the MLWE constituent; covered by [37,19].	ML-KEM library implements constant-time implicit rejection [31].
φ_i^{dom} (<i>domain-label presence</i>)	Confirm the fixed string "Panther/v1" (or the combiner-specific label) appears in the KDF input before any ciphertext byte.	Cross-protocol confusion: output of one combiner reused as input to another, producing collision on ss_h .	All six combiners prepend the combiner-specific label unconditionally (see the Encaps algorithms in Section 4).
φ_i^{ping} (<i>inter-session freshness</i>)	Compare $(\text{ct}_{h,i}, \text{ct}_{1,i}, \text{ct}_{2,i})$ against the previous session; each triple must differ.	Two sessions with identical ciphertexts derive identical ss_h , enabling cross-session key reuse.	Honest party sampling of fresh r_1, r_2 per session makes the ping predicate evaluate to 1 with probability $\geq 1 - 2^{-\gamma}$.
φ_i^{stag} (<i>Panther-SS-only</i>)	Recompute $\text{stag} = ("SS/v1", n, \text{ct}_1 , \text{ct}_2)$ from the received ciphertext; compare byte-for-byte against the stag field.	Structural forgery: adversary re-segments σ under a different splice factor, obtaining a different KDF output.	Panther-SS .Decaps performs this check before consulting the KDF; failure triggers immediate abort.

Table 5: Dominant ask-price contributions (ROM) for each Panther combiner, compared against the two constituent KEMs in isolation. The stand-alone rows are the well-known FO-transform bounds of [31,20,3]; the hybrid rows add combiner-specific statistical terms but crucially require *both* assumptions to be simultaneously broken for total compromise—this is the qualitative benefit of hybridisation.

Scheme / Combiner	Dominant terms in $\text{Ask}(g_{\text{CCA}})$
ML-KEM-768 (alone)	$\text{Adv}^{\text{MLWE}} + q_D 2^{-\gamma_2}$
FrodoKEM-976 (alone)	$\text{Adv}^{\text{LWE}} + q_D 2^{-\gamma_1}$
Panther-P	$\text{Adv}^{\text{dPRF}} + \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + q_D 2^{-\gamma_1} + q_D 2^{-\gamma_2}$
Panther-S	$q_H^2/2^c + \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + q_D 2^{-\gamma_1} + q_D 2^{-\gamma_2}$
Panther-C	$2\text{Adv}^{\text{PRF}} + \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + q_D 2^{-\gamma_1} + q_D 2^{-\gamma_2}$
Panther-X	$2q_H^2/2^c + \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + q_D 2^{-\gamma_1} + q_D 2^{-\gamma_2}$
Panther-N	$\text{Adv}^{\text{dPRF}} + \min(\text{Adv}^{\text{MLWE}}, \text{Adv}^{\text{LWE}}) + q_D 2^{-\gamma_1} + q_D 2^{-\gamma_2}$
Panther-SS	$2(\text{Adv}^{\text{MLWE}} + \text{Adv}^{\text{LWE}}) + q_H^2/2^{c-1} + q_D 2^{-\gamma_1} + q_D 2^{-\gamma_2}$

Table 6: Published side-channel attacks against the two constituent KEMs at NIST Level 3. *Traces* denotes the approximate number of traces/queries for a successful key recovery; *Leakage model* denotes the assumed side-channel oracle. References cited in-text.

Attack	Target	Leakage model	Traces/queries	ML-KEM / FrodoKEM
FO-transform timing attack [19]	decaps re-enc	timing	$\sim 2^{30}$	both
Flush+Reload on NTT [37]	basemul	cache	$10^3\text{--}10^4$	ML-KEM
Single-trace EM attack [36]	decaps	EM	1	ML-KEM
Correlation power analysis [46]	matrix mul.	power	$10^3\text{--}10^4$	FrodoKEM
EM attack on impl. rejec- tion [16]	decaps branch	EM	10^3	both
Fault-induced rej. bypass [45]	rejection seed	voltage glitch	1–10	both

Table 7: Attack-vs-defence matrix. “Stand-alone” columns show whether the published attack against that primitive’s reference implementation succeeds. “Hardened Panther” shows the attack outcome when the same primitive is used inside a hardened Panther combiner (any variant). “Panther-SS” shows the Panther-SS-specific outcome, which is strictly stronger than any other combiner due to the structural-tag hop.

Attack	Stand-alone ML-KEM	Stand-alone FrodoKEM	Hardened Panther	Panther-SS
FO timing [19]	✗	✗	✓	✓
Cache Flush+Reload [37]	✗	n/a	✓	✓
Single-trace EM [36]	✗	n/a	partial	✓
Correlation power [46]	n/a	✗	✓	✓
EM on rejection [16]	✗	✗	✓	✓
Fault-induced bypass [45]	✗	✗	✓	✓
Cross-combiner confusion	n/a	n/a	✗	✓

Table 8: Mean operation latency (μ s) with 95% confidence interval half-width in parentheses; $n = 10^5$ trials. *KeyGen* executes both constituent *KeyGen* calls; it does not depend on the combiner.

Scheme / Combiner	KeyGen	Encaps	Decaps
ML-KEM-768 (baseline)	28 (± 0.3)	39 (± 0.4)	36 (± 0.4)
FrodoKEM-976 (baseline)	2 940 (± 18)	3 310 (± 22)	3 265 (± 20)
Panther-P (parallel HKDF)	2 968 (± 18)	3 354 (± 22)	3 307 (± 20)
Panther-S (SHAKE256 split)	2 968 (± 18)	3 352 (± 21)	3 304 (± 20)
Panther-C (sequential chain)	2 968 (± 18)	3 357 (± 22)	3 311 (± 21)
Panther-X (robust XOR)	2 968 (± 18)	3 358 (± 22)	3 309 (± 20)
Panther-N (robust nested)	2 968 (± 18)	3 363 (± 22)	3 316 (± 21)
Panther-SS (structural splicing)	2 968 (± 18)	3 361 (± 22)	3 313 (± 21)

Table 9: Combiner-only overhead (μ s) for encaps and decaps, after subtracting the baseline FrodoKEM-976 and ML-KEM-768 times. The overhead corresponds to KDF calls, domain-separation tagging, and structural-tag computation.

Scheme / Combiner	Enc overhead	Dec overhead	Δ vs Panther-P
ML-KEM-768 (alone)	0.0	0.0	(no combiner)
FrodoKEM-976 (alone)	0.0	0.0	(no combiner)
Panther-P	5.1	6.3	baseline
Panther-S	3.2	3.5	$-1.9 / -2.8$
Panther-C	8.2	10.1	$+3.1 / +3.8$
Panther-X	9.3	7.7	$+4.2 / +1.4$
Panther-N	14.1	11.3	$+9.0 / +5.0$
Panther-SS	12.0	11.8	$+6.9 / +5.5$

Table 10: Combiner overhead as a fraction of total hybrid latency, compared against the two constituent KEMs in isolation. The denominator for hybrid rows is their own Encaps+Decaps total; the standalone KEM rows have no combiner overhead by construction.

Scheme / Combiner	Total Enc+Dec (μ s)	Combiner overhead (μ s)	Fraction
ML-KEM-768 (alone)	75	0.0	0%
FrodoKEM-976 (alone)	6 575	0.0	0%
Panther-P	6 661	11.4	0.17%
Panther-S	6 656	6.7	0.10%
Panther-C	6 668	18.3	0.28%
Panther-X	6 667	17.0	0.26%
Panther-N	6 679	25.4	0.38%
Panther-SS	6 674	23.8	0.36%

Table 11: Public-key, ciphertext, and shared-secret sizes (bytes) of the two constituent KEMs in isolation versus the six Panther hybrid combiners. All Panther combiners concatenate the two constituent ciphertexts; only Panther-SS adds a 1-byte splice factor.

Scheme / Combiner	Public key	Ciphertext	Shared secret
ML-KEM-768 (alone)	1 184	1 088	32
FrodoKEM-976 (alone)	15 632	15 792	24
Panther-P	16 816	16 880	32
Panther-S	16 816	16 880	32
Panther-C	16 816	16 880	32
Panther-X	16 816	16 880	32
Panther-N	16 816	16 880	32
Panther-SS	16 816	16 881	32

Table 12: Peak per-session memory footprint (KB) at NIST Level 3, comparing the two constituent KEMs in isolation with the six Panther hybrid combiners. The *KEM state* column shows primitive-local stack usage; the *Combiner state* column is non-zero only for hybrid combiners.

Scheme / Combiner	KEM state	Combiner state	Total
ML-KEM-768 (alone)	8.6	0.0	8.6
FrodoKEM-976 (alone)	55.6	0.0	55.6
Panther-P	64.2	0.9	65.1
Panther-S	64.2	0.3	64.5
Panther-C	64.2	1.2	65.4
Panther-X	64.2	0.6	64.8
Panther-N	64.2	1.4	65.6
Panther-SS	64.2	1.1	65.3

Table 13: Benchmark of NIST PQC KEM candidates at NIST Level 3 versus the six Panther hybrid combiners. *Round* refers to the NIST PQC standardisation round in which the scheme last appeared. Values are single-threaded on an Intel Core i7-13700K @ 3.4 GHz using `liboqs` AVX2 paths where available; code-based candidates are reported from their reference implementations at matching security level. Shaded rows are the Panther family.

Scheme (Level 3)	Family	Round	Status	Enc+Dec (μ s)	pk (B)	ct (B)	PK+CT (B)
Classic McEliece (mceliece460896)	code (Goppa)	4	finalist	12 500	524 160	188	524 348
BIKE-L3	code (QC-MDPC)	4	alt	12 000	3 083	3 115	6 198
HQC-192	code (QC)	4	selected	7 000	4 522	8 978	13 500
NTRU (hrss701)	lattice (NTRU)	3	dropped	180	1 138	1 138	2 276
NTRU Prime (sntrup761)	lattice (NTRU)	3	dropped	220	1 158	1 039	2 197
SABER (firesaber)	lattice (MLWR)	3	dropped	130	1 312	1 472	2 784
Kyber 768 $\equiv$ ML-KEM-768	lattice (MLWE)	3	standard	75	1 184	1 088	2 272
FrodoKEM-976	lattice (LWE)	3	alt	6 575	15 632	15 792	31 424
Panther-P(Frodo+ML-KEM, parallel)	hybrid	–	this work	6 661	16 816	16 880	33 696
Panther-S(Frodo+ML-KEM, SHAKE)	hybrid	–	this work	6 656	16 816	16 880	33 696
Panther-C(Frodo+ML-KEM, chained)	hybrid	–	this work	6 668	16 816	16 880	33 696
Panther-X(Frodo+ML-KEM, robust XOR)	hybrid	–	this work	6 667	16 816	16 880	33 696
Panther-N(Frodo+ML-KEM, nested)	hybrid	–	this work	6 679	16 816	16 880	33 696
Panther-SS(Frodo+ML-KEM, spliced)	hybrid	–	this work	6 674	16 816	16 881	33 697

Table 14: Composite security-per-byte-per-microsecond efficiency index $E = \text{Sec}_{\text{bits}} / (\text{Lat}_{\mu\text{s}} \cdot \text{BW}_{\text{KB}})$ for each NIST Level-3 candidate. Higher E means more security delivered per unit bandwidth per unit time. The Panther hybrids sacrifice raw E (because they are *compositions*) but gain $\vee$ -security and side-channel amplification—which this index does not capture.

Scheme (Level 3)	Sec (bits)	Lat (μ s)	BW (KB)	E
Kyber 768 / ML-KEM-768	192	75	2.22	1.15
SABER (firesaber)	192	130	2.72	0.54
NTRU (hrss701)	192	180	2.22	0.48
NTRU Prime (sntrup761)	192	220	2.15	0.41
HQC-192	192	7 000	13.18	2.1×10^{-3}
BIKE-L3	192	12 000	6.05	2.6×10^{-3}
FrodoKEM-976	192	6 575	30.69	9.5×10^{-4}
Classic McEliece (460896)	192	12 500	512.06	3.0×10^{-5}
Panther-P (Frodo+ML-KEM)	192*	6 661	32.91	8.8×10^{-4}
Panther-S (Frodo+ML-KEM)	192*	6 656	32.91	8.8×10^{-4}
Panther-SS (Frodo+ML-KEM)	192*	6 674	32.91	8.8×10^{-4}

Index

- CNF verification, 26
- correctness, 18
- delta-correctness, 18
- HKDF, 11
- implicit rejection, 29
- Keccak, 11
- measure-and-reprogram, 27
- nested KEM, 13
- NIST PQC, 38
- one-way-to-hiding lemma, 27
- Panther-C, 12
- Panther-N, 13
- Panther-P, 11
- Panther-S, 11
- Panther-X, 12
- sequential chaining, 12
- session pinging, 26
- SHAKE256, 11
- side-channel attack, 29
- timing attack, 29
- unbounded security, 26
- XOR combiner, 12